

Every question you have to write, answered in full

What is in here

49 questions and 64 worked answers, from 13 papers and practice sets: the examiner's own past papers, the mock papers, and every written question in the Learn by doing section. Each one is quoted as printed, broken down, then answered in full.

The objective half lives in the [question book](#).

Learn by Doing: Architecture versus Organization

Topic One

Part B: every past question the examiner has set on this topic

Question One: 2025/2026, Q1(a)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 1(a)

THE COURSE TEXT, UNEDITED

a. i) Computer Systems are designed using two closely related but distinct concepts which are Computer Architecture and Computer Organization. Differentiate between these concepts.

2mks

ii) Compare Von Neumann Architecture and Harvard Architecture. Include a diagram for each.

3mks

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

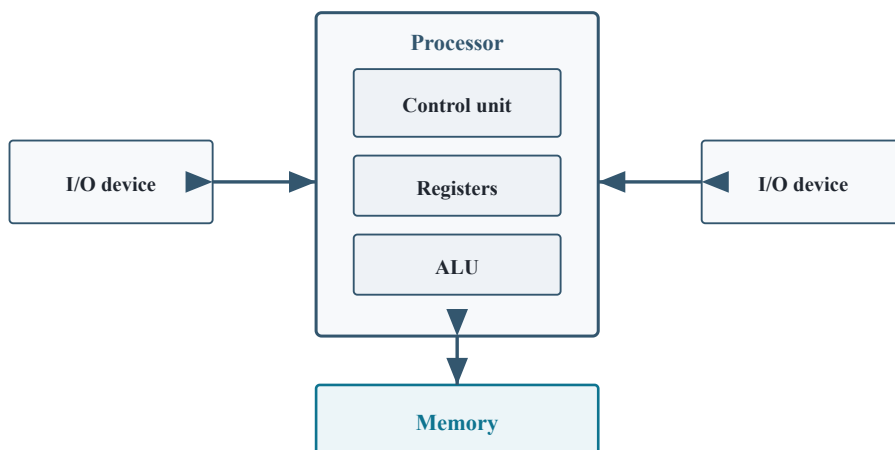
i) Differentiate computer architecture from computer organization.

Computer architecture is defined as the functional design and logical structure of the computer system as the programmer sees it: the instruction set, the data types, the addressing modes and the register organization. It answers the question "*What does the computer do?*" and serves as the blueprint for the system.

Computer organization is defined as the operational units and their interconnections that realize (carry out) that architecture: the control signals, the buses, the memory technology used and the cache. It answers the question "*How does the computer do it?*" A useful likeness: architecture is the building's design, organization is the actual construction of the building.

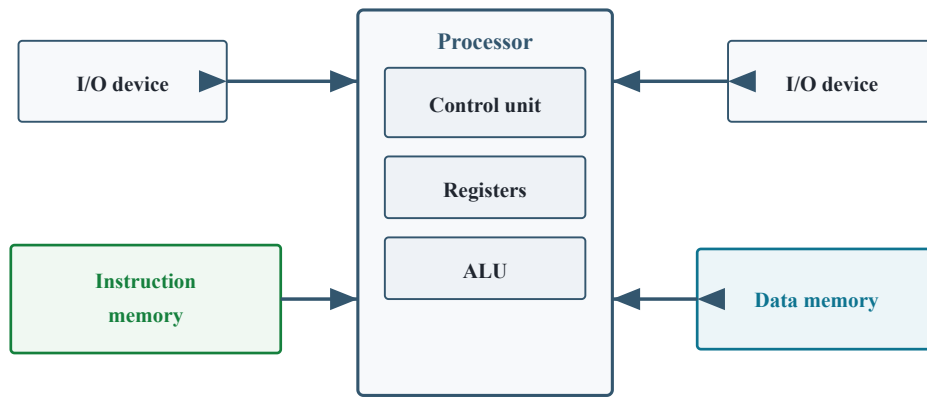
ii) Compare Von Neumann and Harvard architecture, with a diagram for each.

Von Neumann architecture is defined as a design that uses a **single** (shared) area of memory, reached over **one** bus, for both the program instructions and the data those instructions act on. It is simple to build, but the shared path is a bottleneck (an instruction fetch and a data access cannot happen at the same instant).



Von Neumann architecture, the layout the lecturer draws: one Memory serves the whole processor over a single shared path (instructions and data together).

Harvard architecture is defined as a design that uses **separate** memories and **separate** buses for instructions and data, which lets it **fetch an instruction and read data at the same time** (a form of parallelism), at the cost of duplicating the address, data and control lines.



Harvard architecture, the lecturer's layout: a separate instruction memory (fetched, one arrow in) and data memory (read and written), each on its own path, so both are reached at once.

Part 2: the method behind these, taught from nothing

Learn by Doing: Signed Numbers and Complements

Topic Four

Part B: every past question the examiner has set on this topic

Question One: 2025/2026, Q1(c)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 1(c), and in 2 other papers

THE COURSE TEXT, UNEDITED

c. A spacecraft telemetry system uses a 16-bit processor to transmit environmental measurements. The following values are to be stored – Temperature = -182; Pressure = 487; Altitude = -1023. The processor uses 2's complement representation for signed integers.

i) Determine the range of values representable in a 16-bit two's complement system.

Indicate whether the given numbers fall within the range of value

2mks

ii) Represent each value in: Binary and Hexadecimal

3mks

iii) Demonstrate how the processor uses two's complement to compute $(-182) + 487$

2mks

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

GIVEN 16-bit two's complement ; Temperature -182 , Pressure 487 , Altitude -1023

FIND the range and whether each fits ; each in binary and hex ; $(-182) + 487$

FORMULA range = -2^{15} to $2^{15} - 1$ · negative = invert the positive and add 1 · group 4 bits per hex digit

Range: -2^{15} to $2^{15} - 1 = -32768$ to 32767 (all three fit)

$487 = 0000\ 0001\ 1110\ 0111 = 01E7$

-182 : $+182 = 0000\ 0000\ 1011\ 0110$
 invert = $1111\ 1111\ 0100\ 1001$
 $+ 1 = 1111\ 1111\ 0100\ 1010 = FF4A$

-1023 : $+1023 = 0000\ 0011\ 1111\ 1111$
 invert = $1111\ 1100\ 0000\ 0000$
 $+ 1 = 1111\ 1100\ 0000\ 0001 = FC01$

$(-182) + 487$:

$1111\ 1111\ 0100\ 1010$
 $+ 0000\ 0001\ 1110\ 0111$

$1\ 0000\ 0001\ 0011\ 0001$ drop the carry off the top
 $0000\ 0001\ 0011\ 0001 = 256+32+16+1 = 305$

Range -32768 to 32767 (all fit). $487 = 01E7$, $-182 = FF4A$, $-1023 = FC01$. $(-182) + 487 = 305$: the machine adds the patterns and gets the right signed answer.

Question Two: 2024/2025, Q3(b)**THE SCENARIO, AS THE COURSE TEXT PRINTS IT**

2024/2025, Question 3(b), and in 2 other papers

b. i) Complete table 1. Show all necessary work done to arrive at the answer. Use 12 bits. **6mks**

Value	Unsigned	S-M	1's Complement	2's Complement
204				
-139				
1101 00110 11				

ii) Calculate the memory requirement of an image whose dimension is 17.12 inches by 9.45 inches at resolution 250 dpi. Express your answer in Megabyte. **3mks**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN two decimal values, 204 and -139, one bit pattern, four representation schemes and a width of 12 bits.

ASKED *complete* the table and "Show all necessary work done to arrive at the answer", which is an explicit instruction that a filled grid with no working will not score. Part (ii) then switches subject entirely to an image size calculation.

MARKS $6 + 3 = 9$. The 6 covers eight or more cells plus the working, so a cell is worth well under a mark: speed matters more than elegance here.

TAUGHT IN Module One Unit 2, Data Representation.

TRAP writing "not representable" and moving on. Unsigned genuinely cannot hold -139, and saying so earns the mark, but every other cell must still be filled. The second trap is

the 11-bit row: pad it to 12 on the left, state that you have done so, and read it in all four schemes.

GIVEN values 204 and $-139 \cdot n = 12 \text{ bits} \cdot \text{four schemes}$

FIND each value in each scheme

FORMULA sign-magnitude = sign bit + magnitude · one's complement = invert all n bits · two's complement = invert then add 1

Step 1: 204, and why it costs one conversion rather than four. $204 = 128 + 64 + 8 + 4$, so in 12 bits $204 = \mathbf{0000\ 1100\ 1100}$. It is positive, and a positive value is written identically in all four schemes, so that row is one conversion copied across four columns. Write that sentence: it is the difference between a filled row and a justified one.

Step 2: -139 , one scheme at a time. Write the magnitude first: $139 = 128 + 8 + 2 + 1$, so $+139 = \mathbf{0000\ 1000\ 1011}$. Then:

- **Unsigned:** not representable. Unsigned has no sign bit, so 12 bits hold 0 to 4095 and nothing negative. Write that in the cell rather than leaving it blank.
- **Sign-magnitude:** keep the magnitude, set the sign bit to 1: $0000\ 1000\ 1011$ becomes **1000 1000 1011**.
- **One's complement:** invert every one of the 12 bits of $+139$, the leading zeros included: $0000\ 1000\ 1011$ inverts to **1111 0111 0100**.
- **Two's complement:** add 1 to the one's complement: $1111\ 0111\ 0100 + 1 = \mathbf{1111\ 0111\ 0101}$.

Step 3: the third row, which is where the marks hide. The paper prints **1101 00110 11**, grouped 4, 5 and 2, and that is **11 bits** while the question says to use 12. Say so in your script, then pad it on the left to twelve: **0110 1001 1011**. It now begins with 0, so it is positive, and a positive pattern reads the same in every scheme: $1024 + 512 + 128 + 16 + 8 + 2 + 1 = \mathbf{1691}$.

Read instead at its own eleven bits, the leading 1 makes it negative and the three signed schemes disagree, which is worth one line each: sign-magnitude -667 , one's complement -356 , two's complement -357 . Either reading earns the marks provided you state which you took and why.

Value	Unsigned	Sign-magnitude	One's complement	Two's complement
204	0000 1100 1100	0000 1100 1100	0000 1100 1100	0000 1100 1100
-139	not representable	1000 1000 1011	1111 0111 0100	1111 0111 0101
1101 00110 11 padded to 0110 1001 1011	1691	+1691	+1691	+1691

Reading 1111 0111 0101 back as two's complement: the leading 1 says negative, so invert and add 1, giving $0000\ 1000\ 1011 = 139$, so the value is **-139** . The recipe reverses itself, which is the check to write down.

Question Three: 2024/2025, Q4(b)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question 4(b), and in 1 other paper

THE COURSE TEXT, UNEDITED

b. i) Describe End-Around Carry in 1's Complement? Illustrate this using $153 - 142$ in a 10-bit register. **4mks** ii) What do you understand by Overflow and Underflow in data representation? Give one example for each. **3mks**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN 153 and 142, a 10-bit register, and one's complement arithmetic. Part (ii) is given nothing at all.

ASKED *describe* end-around carry and then *illustrate* it, so a worked subtraction with no explanation, or an explanation with no subtraction, each lose half. Part (ii) wants two definitions and one example each, four items.

MARKS $4 + 3 = 7$. The example in (ii) is worth as much as the definition, so never skip it.

TAUGHT IN Module One Unit 2, Data Representation.

TRAP discarding the carry. In one's complement the carry out of the top bit is added back into the lowest bit, and that end-around step is the entire point of the question; in two's complement the same carry is thrown away. Mixing the two rules gives an answer one short.

GIVEN $153 - 142$ · $n = 10$ bits · one's complement

FIND the difference as a pattern and in decimal

FORMULA $a - b = a + (-b)$ with $-b$ the bitwise inversion; a carry out of the top is **added back** into the lowest bit

$153 = 00\ 1001\ 1001$; $+142 = 00\ 1000\ 1110$ so $-142 = 11\ 0111\ 0001$. Adding: $00\ 1001\ 1001 + 11\ 0111\ 0001 = 1\ 00\ 0000\ 1010$, with a carry out. End-around: $00\ 0000\ 1010 + 1 = \mathbf{00\ 0000\ 1011 = 11}$, and $153 - 142 = 11$. Correct. (ii) **Overflow** is a result too large for the available bits (in 8-bit two's complement, $100 + 50 = 150$ overflows the $+127$ limit). **Underflow** is a result too small to represent: for integers more negative than the smallest storable value ($-100 - 50 = -150$ below -128), and for floating point a value so close to zero it rounds away to nothing.

Question Four: 2023/2024, Q4(b)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2023/2024, Question 4(b), and in 2 other papers

THE COURSE TEXT, UNEDITED

b. i) Complete the table below. Show all necessary work done to arrive at the answer. Use 12bits. **6mks**

Value	Unsigned	S-M	1's Complement	2's Complement
204				
-139				
1101 00110 11				

BREAK IT DOWN FIRST

before you write a word

What is this question actually asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given two decimal values, 204 and -139 , one bit pattern, 1101 00110 11, and a register width of 12 bits. Four columns are wanted for each row: Unsigned, S-M, one's complement, two's complement.

Asked two things joined by a full stop: *complete the table*, and *show all necessary work done to arrive at the answer*. The second sentence is worth marks on its own, so a page of finished bit patterns with no derivation is a half answer.

Marks 6mks across twelve cells, so no single cell is worth arguing over. The working is what separates a 3 from a 6.

Taught in this unit, Module One Unit 2.

Trap the 11-bit row. Do not silently pad it to 12 and say nothing, and do not stall. Write one line saying the row as printed is 11 bits against an instruction of 12, state the assumption you are making (a leading 0), and carry on. Second trap: 204 is positive, so it is identical in all four columns; only the negative row needs real work.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Answer **2023/2024 Q4(b)** in full in your book, then open this and mark yourself against it.

GIVEN values = 204 and $-139 \cdot n = 12$ bits $\cdot$ four schemes required

FIND each value's bit pattern in every scheme

FORMULA sign-magnitude: sign bit + magnitude $\cdot$ one's complement: invert all n bits $\cdot$ two's complement: invert, then add 1

WHAT EACH SYMBOL MEANS

$n = 12$, the total bits *including* the sign bit $\cdot$ a **positive** value looks identical in all four schemes, so only the negative needs work $\cdot$ "invert all n bits" includes the leading zeros

1. Write 204 in 12-bit binary. $204 = 128 + 64 + 8 + 4$, so $204 = 0000\ 1100\ 1100$. Being positive, 204 looks identical in all four schemes.
2. For -139 , first write $+139$ in 12 bits: $139 = 128 + 8 + 2 + 1 = 0000\ 1000\ 1011$.
3. Sign-magnitude of -139 : set the sign bit to 1, keep the magnitude: $1000\ 1000\ 1011$.
4. One's complement of -139 : invert every bit of $0000\ 1000\ 1011$: $1111\ 0111\ 0100$.
5. Two's complement of -139 : add 1 to the one's complement: $1111\ 0111\ 0100 + 1 = 1111\ 0111\ 0101$.

Value	Unsigned	Sign-magnitude	One's complement	Two's complement
204	0000 1100 1100	0000 1100 1100	0000 1100 1100	0000 1100 1100
-139	not representable	1000 1000 1011	1111 0111 0100	1111 0111 0101

Now redo it: the third row on the real paper gives a bit pattern and asks its value. Read back $1111\ 0111\ 0101$ as two's complement: the leading 1 says negative, so invert and add 1 ($0000\ 1000\ 1011 = 139$), giving -139 . That is the same number, proving the recipe reverses itself.

Question Five: 2020/2021, Q5(a)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2020/2021, Question 5(a), and in 1 other paper

THE COURSE TEXT, UNEDITED

ii) Describe End-Around Carry in 1's Complement? Illustrate this using 153 -

142 in an 10-bit register.

4mks

BREAK IT DOWN FIRST

before you write a word

What is this question actually asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given the operands 153 and 142, a register 10 bits wide, and the scheme named for you as one's complement.

Asked two verbs. *Describe* what end-around carry is, then *illustrate* it on these particular numbers. Marks sit behind both.

Marks 4mks, which splits sensibly as one for the description and three for the illustration, since the illustration is five lines of working.

Taught in this unit, Module One Unit 2.

Trap reaching for two's complement out of habit and throwing the carry away. In one's complement the carry comes back in at the bottom, and discarding it here leaves you with 10 instead of 11, wrong by exactly one and wrong in a way that looks right. Second trap: writing 153 and 142 in eight bits because they fit. The question says 10, so pad both to 10 before inverting, or the inversion produces the wrong pattern.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Answer **2020/2021 Q5(a)** in full in your book, then open this and mark yourself against it.

GIVEN minuend = 153 · subtrahend = 142 · n = 10 bits · scheme = one's complement

FIND the difference, as a 10-bit pattern and in decimal

FORMULA $a - b = a + (-b)$, where $-b$ is the bitwise inversion of b ; then if a carry leaves the top, **add it back** into the lowest bit

WHAT EACH SYMBOL MEANS

n = 10, the register width, so every pattern is written to 10 bits · **end-around carry** = that add-it-back step, and it happens *only* in one's complement · no carry out means no correction, and the result is negative

1. $153 - 142$ becomes $153 + (-142)$. Write 153 in 10 bits: 00 1001 1001.
2. Write -142 in one's complement: $+142 = 00\ 1000\ 1110$, invert to 11 0111 0001.
3. Add: $00\ 1001\ 1001 + 11\ 0111\ 0001 = 1\ 00\ 0000\ 1010$. A 1 has carried out of the 10th bit.
4. End-around carry: drop that carry and add it into the lowest bit: $00\ 0000\ 1010 + 1 = 00\ 0000\ 1011$.
5. Read the result: $00\ 0000\ 1011 = 11$. Positive (leading 0), and $153 - 142 = 11$. Correct.

$$153 - 142 = 00\ 0000\ 1011 \text{ (one's complement, 10 bits)} = 11.$$

Now redo it: the signature of one's complement is that a carry out never vanishes, it comes back in at the bottom. If there is no carry out, there is nothing to add and the result is already negative.

Question 6: 2020/2021, Q5(b)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2020/2021, Question 5(b)

THE COURSE TEXT, UNEDITED

ii) What will be displayed (in binary and decimal) as the sum of 214 and 65 using an 8-bit register? Show details of work? **3mks**

BREAK IT DOWN FIRST

before you write a word

What is this question actually asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given two decimal values, 214 and 65, and a register 8 bits wide. Nothing is said about signed or unsigned.

Asked what the register *displays*, given *in binary and decimal*, with the working shown. That is three deliverables, not one.

Marks 3mks, which reads naturally as one for the binary pattern, one for its decimal value, one for the working that gets you there.

Taught in this unit, Module One Unit 2.

Trap writing 279. That is the arithmetic answer, and the question did not ask for it. Give the 8 bits the register actually holds, their decimal value, and one sentence naming what happened to the ninth bit. Second trap: answering in decimal only when the paper asked for both forms.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Answer **2020/2021 Q5(b)** in full in your book, then open this and mark yourself against it.

GIVEN operands = 214 and 65 · n = 8 bits · unsigned

FIND the stored result in binary and decimal, and whether it overflows

FORMULA unsigned capacity = $2^n - 1$ · a register keeps only the low n bits, so the stored value is the true sum mod 2^n

WHAT EACH SYMBOL MEANS

$n = 8$, so the register holds 0 to 255 · **mod 2^n** means the true sum wraps by exactly 256 · **overflow** = the true answer needed more bits than the register has, signalled by a carry out of the top

1. Convert: $214 = 1101\ 0110$, $65 = 0100\ 0001$.
2. Add the two bytes: $1101\ 0110 + 0100\ 0001 = 1\ 0001\ 0111$. The result needs 9 bits.
3. An 8-bit register holds only the low 8 bits, $0001\ 0111 = 23$, and sets a carry.

True sum $214 + 65 = 279$. In 8 bits the register keeps $0001\ 0111 = 23$ and raises a carry out of the top bit. This is unsigned **overflow**: 279 will not fit in a byte (max 255), so the stored 23 is wrong by exactly $2^8 = 256$.

Now redo it: overflow is simply "the answer needed more bits than the box has". State the true sum, the wrapped value, and that a carry came out.

Part 2: the method behind these, taught from nothing

Learn by Doing: Floating Point and IEEE 754

Topic Five

Part B: every past question the examiner has set on this topic

Question One: 2025/2026, Q4(a)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 4(a), and in 2 other papers

THE COURSE TEXT, UNEDITED

- a. Consider three (3) floating point numbers X , Y and Z stored in registers based on the IEEE 754 Single precision floating point format.
 $X = C1400000h$, $Y = 42100000h$, $Z = 41400000h$
Convert X , Y and Z to decimal numbers and determine which of the following is true. **9mks**
- i. $(Y-Z) > 0$
 - ii. $X + Z = 0$
 - iii. $Z = X + Y$

BREAK IT DOWN FIRST

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

GIVEN $X = \text{C1400000h}$, $Y = \text{42100000h}$, $Z = \text{41400000h}$, IEEE-754 single precision

FIND each in decimal, then which claim is true

FORMULA $\text{value} = (-1)^s \times 1.M \times 2^{(E - 127)}$, where s = sign bit, M = mantissa, E = exponent field

$X = \text{C1400000} = 1\ 10000010\ 100\ 0000\dots$
 sign 1 (neg); exp 130, $e = 3$; $1.100\dots = 1.5$
 $X = -1.5 \times 2^3 = -12$

$Y = \text{42100000} = 0\ 10000100\ 001\ 0000\dots$
 sign 0; exp 132, $e = 5$; $1.001\dots = 1.125$
 $Y = 1.125 \times 2^5 = 36$

$Z = \text{41400000} = 0\ 10000010\ 100\ 0000\dots$
 sign 0; exp 130, $e = 3$; 1.5
 $Z = +1.5 \times 2^3 = 12$

$(Y - Z) = 36 - 12 = 24 > 0$ TRUE
 $X + Z = -12 + 12 = 0$ TRUE
 $Z = X + Y$? $12 = -12 + 36 = 24$ FALSE

$X = -12$, $Y = 36$, $Z = 12$. (i) true, (ii) true, (iii) false.

Question Two: 2025/2026, Q5(a)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 5(a), and in 1 other paper

THE COURSE TEXT, UNEDITED

a. Represent the following -1240.56 using Single Precision IEEE format for floating point numbers. **6mks**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN -1240.56, single precision

FIND the 32-bit pattern (and its hex)

FORMULA sign ; normalise $|x|$ to $1.M \times 2^e$; exp field = $e + 127$; mantissa $M = f$ to 23 bits

sign = 1 (negative)

1240.56 in binary = $1.0011011000100011110101... \times 2^{10}$

exp field = $10 + 127 = 137 = 1000\ 1001$

mantissa (23 bits) = 0011011 0001000 1111010 11

pattern = 1 10001001 00110110001000111101011

= C49B11EB (truncated; C49B11EC if rounded)

-1240.56 = **C49B11EB** (truncated), or C49B11EC rounded. State which.

Question Three: 2025/2026, Q5(d)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 5(d), and in 1 other paper

THE COURSE TEXT, UNEDITED

d. By using the reverse of the IEEE Single Precision procedure, determine the value (Base 10) of the following hexadecimal (base 16) value: C4F2E000h

6mks

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN C4F2E000h, single precision

FIND the decimal value

FORMULA $\text{value} = (-1)^s \times 1.M \times 2^{(E - 127)}$, where s = sign bit, M = mantissa, E = exponent field

C4F2E000 = 1 10001001 1110010111000000000000

sign 1 (neg); exp field 1000 1001 = 137, $e = 137 - 127 = 10$

$1.M = 1.1110010111 = 1.8974609375$

$\text{value} = -1.8974609375 \times 2^{10} = -1.8974609375 \times 1024$
 $= -1943$

C4F2E000h = -1943.

Question Four: 2024/2025, Q1(a)

Question Five: 2024/2025, Q3(c)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question 3(c), and in 1 other paper

THE COURSE TEXT, UNEDITED

c. Represent the following -1240.56 using Single Precision IEEE format for floating

point numbers. 6mks

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN one decimal value, -1240.56, and the target format, IEEE single precision.

ASKED *represent* it. The verb wants the 32-bit pattern, and by convention the hexadecimal equivalent alongside it.

MARKS 6, which is the whole of a five part question's third part. Expect a mark each for the sign, the integer part in binary, the fraction in binary, the normalisation, the biased exponent and the assembled pattern.

TAUGHT IN Module One Unit 3, Floating Point Representation.

TRAP the fraction. 0.56 never terminates in binary, so it must be expanded by repeated doubling until the mantissa is full, and then rounded or truncated. Say which you did: the last hex digit changes.

Sign 1. 1240 = 100 1101 1000 and 0.56 = 0.10001111010111... Normalising gives $1.001101100010001111010111... \times 2^{10}$, so $E = 10 + 127 = 137 = 1000\ 1001$, and the first 23 mantissa bits are 0011 0110 0010 0011 1101 011. Pattern 1 10001001 00110110001000111101011 = **C49B11EBH** (truncated). Rounding the last bit instead gives C49B11ECH, which is what hardware stores; either is accepted if you state the method.

Question 6: 2024/2025, Q4(c)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question 4(c), and in 1 other paper

THE COURSE TEXT, UNEDITED

c. By using the reverse of the IEEE Single Precision procedure, determine the value (Base 10) of the following hexadecimal (base 16) value: C4F2E00h **6mks**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN one 32-bit hexadecimal word and the statement that it is IEEE single precision.

ASKED *determine* the base 10 value, by running the conversion procedure backwards. The words "Base 10" and "base 16" are spelled out so that no answer in binary or hex will do.

MARKS 6. Roughly one each for the sign, the exponent field, the unbiased exponent, restoring the hidden 1, the shifted binary value and the decimal total.

TAUGHT IN Module One Unit 3, Floating Point Representation.

TRAP forgetting the hidden bit. The stored mantissa is the fraction after an implied leading 1, so 1.11100101110 is what you shift, not 0.11100101110. Miss it and the answer halves.

C4F2E000 = 1100 0100 1111 0010 1110 0000 0000 0000. Sign 1, so negative. E = 1000 1001 = 137, so e = 137 - 127 = 10. Restoring the hidden 1 gives $1.11100101110 \times 2^{10} = 11110010111_2 = 1024 + 512 + 256 + 128 + 16 + 4 + 2 + 1 = 1943$. So the value is **-1943**.

Question 7: 2024/2025, Q5(c)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question 5(c)

THE COURSE TEXT, UNEDITED

c. Perform the calculation below in using Single Precision IEEE format. All bits after the first 12 bits of the Mantissa should be 0. -421.55 (all operations must be done in binary). **5½mks**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN the value -421.55, the single precision format, and a truncation rule: everything after the first 12 mantissa bits is zero.

ASKED the 32-bit pattern, in binary, with the mantissa truncated as instructed. The truncation is not a simplification you are being offered, it is a requirement you are being marked on.

MARKS 5½. Sign, integer part, fraction, normalisation, exponent field and the truncated assembly, so about a mark each with a half for the unit of the answer.

TAUGHT IN Module One Unit 3, Floating Point Representation.

TRAP truncating at the wrong place. "the first 12 bits of the Mantissa" means the 23-bit stored fraction, not the 24 bits including the hidden 1, so you keep 12 fraction bits and set the remaining 11 to zero. Say what the truncated pattern now reads back as: that is what the question is testing.

GIVEN value = -421.55 · IEEE-754 single precision · keep only the first **12** mantissa bits, rest zero

FIND the full 32-bit pattern in binary

FORMULA normalise to $1.m \times 2^e$, $E = e + 127$, then truncate the mantissa as instructed

1. Sign: negative, so **1**.

2. $421 = 1\ 1010\ 0101$, and $0.55 = 0.100011001100\dots$ so $421.55 = 110100101.100011001100\dots$

3. Normalise: $1.10100101100011001100110 \times 2^8$, so $e = 8$.

4. Exponent field: $8 + 127 = 135 = \mathbf{1000\ 0111}$.

5. Mantissa, first 12 bits: 1010 0101 1000, then eleven zeros: **1010 0101 1000 0000 0000 000**.

$-421.55 \approx \mathbf{1\ 10000111\ 101001011000000000000000} = \mathbf{C3D2C000H}$. Truncating at 12 mantissa bits loses precision: this pattern reads back as -421.5 exactly, against the untruncated $\mathbf{C3D2C666H}$. Say so, because the question is testing that you know truncation costs accuracy.

Question 8: 2023/2024, Q1(c)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2023/2024, Question 1(c), and in 2 other papers

THE COURSE TEXT, UNEDITED

c. Consider three (3) floating point numbers X, Y and Z stored in registers based on the IEEE 754 Single precision floating point format.
 $X = \mathbf{C1400000h}$, $Y = \mathbf{42100000h}$, $Z = \mathbf{41400000h}$
Convert X, Y and Z to decimal numbers and determine which of the following is true or false **9mks**

- i. $(Y-Z) > 0$
- ii. $X + Z = 0$
- iii. $Z = X + Y$

BREAK IT DOWN FIRST

before you write a word

What is this question actually asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given three 32-bit hex words and the statement that they are IEEE 754 *single* precision. That one word fixes the split at 1 sign bit, 8 exponent bits and 23 mantissa bits, and the bias at 127.

Asked two instructions in one sentence: *convert* all three to decimal, then *determine* the standing of three claims. The second verb carries the marks. A script that converts perfectly and stops has answered half the question.

Marks 9mks, the biggest single part anywhere in this unit, split across three conversions and three verdicts. Roughly 2 per conversion and 1 per verdict, so the conversions must be shown rather than asserted.

Taught in this unit, Module One Unit 3.

Trap letting the year's wording decide how many verdicts you give. Write TRUE or FALSE against every one of the three, every time, and you satisfy both versions of the question. Second trap:

comparing the hex patterns directly. They are not ordinary integers; convert first, compare afterwards.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Answer **2023/2024 Q1(c)** in full in your book, then open this and mark yourself against it.

GIVEN $X = \text{C1400000H}$ · $Y = \text{42100000H}$ · $Z = \text{41400000H}$ · all IEEE-754 single precision, bias 127

FIND the decimal value of each, then the truth of the three statements

FORMULA $\text{value} = \pm 1.M \times 2^{(E - 127)}$, then ordinary arithmetic on the decimals

WHAT EACH SYMBOL MEANS

E = stored exponent field of that pattern · **M** = its mantissa bits · convert **first**, compare **second**: the hex patterns cannot be compared directly

Convert each

Hex	Sign / Exp field / e	1.mantissa $\times 2^e$	Decimal
C1400000	1 / 130 / 3	$-1.1 \times 2^3 = -1100$	-12
42100000	0 / 132 / 5	$1.0001 \times 2^5 = 100100$	36
41400000	0 / 130 / 3	$1.1 \times 2^3 = 1100$	12

Apply the tests

$X = -12, Y = 36, Z = 12$. (i) $Y - Z = 36 - 12 = 24 > 0$: **TRUE**. (ii) $X + Z = -12 + 12 = 0$: **TRUE**.
(iii) $X + Y = -12 + 36 = 24$, which is not 12, so $Z = X + Y$ is **FALSE**.

Now redo it: convert all three first, then the three tests are one line of ordinary arithmetic each. Do not try to compare the hex directly.

Part 2: the method behind these, taught from nothing

Learn by Doing: Instruction Types and Formats

Topic Six

Part B: every past question the examiner has set on this topic

Question One: 2025/2026, Q4(b)

THE COURSE TEXT, UNEDITED

b. Use a Zero, One, Two and Three address instruction formats to write a Pentium 4 assembly language program segment for the expression below. For each format determine the number of bytes required.

$$X = ((A * B) + C/D) / (F * G + J * K)$$

8mks

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

On the sheet the sentence "For each format determine the number of bytes required" is struck through in pen, so in that one sitting only the four programs were wanted. But the PRINTED paper asks for the byte counts too, so here is both: the programs, then the byte count for each format. If the strike is not on your paper, you have the full answer; if it is, the totals cost four lines and cannot lose marks.

Three-address (numerator, then denominator, then divide):

```
MUL  R1, A, B      ; R1 = A*B
DIV  R2, C, D      ; R2 = C/D
ADD  R1, R1, R2    ; numerator
MUL  R2, F, G
MUL  R3, J, K
ADD  R2, R2, R3    ; denominator
DIV  X, R1, R2     ; X = numerator / denominator
```

Two-address: MOV/op with the destination doubling as a source. **One-address:** work through the accumulator (LOAD, op, STORE). **Zero-address:** a stack (PUSH the eight operands, then each ADD/MUL/DIV pops two and pushes the result, finally POP X).

Byte count for each format (the printed part, worked)

GIVEN the paper gives no field sizes, so state the usual convention: opcode = 1 byte, each address = 2 bytes

FORMULA bytes per instruction = opcode + (address fields × 2) · total = instruction count × bytes per instruction

Format	Instructions	Bytes
3-address	7	$7 \times (1 + 3 \times 2) = 7 \times 7 = \mathbf{49}$
2-address	12	$12 \times (1 + 2 \times 2) = 12 \times 5 = \mathbf{60}$
1-address	17	$17 \times (1 + 1 \times 2) = 17 \times 3 = \mathbf{51}$
0-address	16	$8 \text{ pushes} \times 3 + 1 \text{ pop} \times 3 + 7 \text{ ops} \times 1 = \mathbf{34}$

Same X in all four formats. Byte totals **49, 60, 51 and 34** for 3, 2, 1 and 0-address. Fewer address fields shortens each instruction but needs more of them, so the totals stay close, and 0-address is smallest here because its operation instructions carry no address at all.

Question Two: 2024/2025, Q1(d)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question 1(d), and in 1 other paper

THE COURSE TEXT, UNEDITED

d. Use a Zero, One, Two and Three address instruction formats to write a Pentium 4 assembly language program segment for the expression below. For each format determine the number of bytes required. **8mks** $X = ((A * B) + C/D)/(F * G + J * K)$

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN one arithmetic expression and four instruction formats to write it in. The paper does not state the opcode or address field sizes, so you state them yourself: 1 byte per opcode and 2 bytes per address is the usual convention and the one used here.

ASKED two things joined by "For each format": *write* the program segment, and *determine* the byte count. Four programs and four totals.

MARKS 8, which is 2 per format. A format with a listing and no byte count is half answered.

TAUGHT IN Module One Unit 4, Instruction Set Architecture.

TRAP assuming fewer address fields always means fewer bytes. It does not: 1-address needs 51 bytes against 2-address at 60 but 3-address at only 49, because shortening each instruction lengthens the program. Count instructions first, then multiply.

GIVEN the expression $\cdot \text{opcode} = 1 \text{ byte} \cdot \text{each address} = 2 \text{ bytes}$

FIND the four programs and their byte totals

FORMULA bytes per instruction = opcode bytes + (address fields $\times$ bytes per address); total = count $\times$ that

Format	Instructions	Bytes
3-address	7	$7 \times (1 + 3 \times 2) = 7 \times 7 = 49$
2-address	12	$12 \times (1 + 2 \times 2) = 12 \times 5 = 60$
1-address	17	$17 \times (1 + 1 \times 2) = 17 \times 3 = 51$
0-address	16	$8 \text{ pushes} \times 3 + 1 \text{ pop} \times 3 + 7 \text{ ops} \times 1 = 34$

The full listings are worked in Module One Unit 4. Totals: **49, 60, 51 and 34 bytes** for 3, 2, 1 and 0 address respectively. Fewer address fields shortens each instruction but needs more of them, which is why the totals stay close.

Question Three: 2024/2025, Q2(c)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question 2(c)

THE COURSE TEXT, UNEDITED

c. Provide an assembly language program to add two numbers and store the result.

4mks

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN nothing at all. Two numbers exist only as names you choose.

ASKED *provide* a program: three operations, load, add, store. "store the result" is a separate demand from "add", so an answer that ends at ADD is incomplete.

MARKS 4, which for three instructions means roughly one per instruction plus one for the assumption you state.

TAUGHT IN Module One Unit 4, Instruction Set Architecture.

TRAP leaving the machine unstated. Write the accumulator assumption on the line above the code and the same three lines become unambiguous. Comment each line: comments cost nothing and show the marker you know what the accumulator holds.

On a one-address (accumulator) machine:

LOAD A ; accumulator = A

ADD B ; accumulator = A + B

STORE C ; C = A + B

On a three-address machine it is the single instruction ADD C, A, B. Either scores full marks provided you state which machine you assume.

Question Four: 2024/2025, Q5(a)**THE SCENARIO, AS THE COURSE TEXT PRINTS IT**

2024/2025, Question 5(a)

THE COURSE TEXT, UNEDITED

- a. i) Describe three (3) categories of Instructions that we have. Give two (2) examples for each. **4½mks** ii) Discuss the characteristics of computer memory based

on Access Methods. Give one example for each method. **4mks**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN nothing numeric. Two recall parts.

ASKED *describe* three instruction categories and *give* two examples of each, six examples in all, then *discuss* memory by access method with *one* example each.

MARKS $4\frac{1}{2} + 4 = 8\frac{1}{2}$, the largest recall block on the paper. That is $1\frac{1}{2}$ per instruction category, so a name with two examples and a sentence of description.

TAUGHT IN Module One Unit 4 for the instruction categories, Module Three Unit 1 for the access methods.

TRAP answering (ii) with RAM, ROM and cache. Those are technologies. The access methods are sequential, direct, random and associative, and each wants its own example device.

(i) **Data transfer:** LOAD, STORE (also MOVE, PUSH, POP). **Arithmetic and logic:** ADD, AND (also SUB, MUL, OR, NOT, shifts). **Control (test and branch):** JUMP, BEQ (also CALL, RETURN).
(ii) **Sequential:** read in order from the start, access time depends on position, as in magnetic tape. **Direct:** jump near the item then search a small region, as on a disk. **Random:** every location takes the same time via its address, as in RAM. **Associative:** located by content rather than address, all locations compared at once, as in a cache tag search.

Part 2: the method behind these, taught from nothing

Learn by Doing: Addressing Modes

Topic Seven

Part B: every past question the examiner has set on this topic

Question One: 2025/2026, Q1(e)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 1(e)

THE COURSE TEXT, UNEDITED

e. A processor contains the following information: PC = 7000; R1 = 1500;

Memory Contents

Address Value

7000 1200

1200 4500

1500 6000

3200 8100

4500 999

6000 200

8100 400

Assume an instruction contains address field 1200.

Determine the Effective Address (EA) and operand for the following addressing modes:

i) Immediate addressing mode 1mk

ii) Direct addressing mode 1mk

iii) Indirect addressing mode 1mk

iv) Register addressing mode 1mk

v) Register Indirect (R1) addressing mode 1mk

vi) Relative addressing mode 1mk

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

GIVEN PC = 7000, R1 = 1500, address field = 1200 ; M[1200]=4500, M[4500]=999, M[1500]=6000

FIND the EA and operand under each mode

FORMULA the mode says how to turn the field into the EA ; the operand is M[EA]

Mode	Effective address	Operand
i) Immediate	none	the field = 1200
ii) Direct	1200	M[1200] = 4500
iii) Indirect	M[1200] = 4500	M[4500] = 999
iv) Register	none	R1 = 1500
v) Register indirect	R1 = 1500	M[1500] = 6000
vi) Relative	PC + 1200 = 8200	M[8200], not in the given table

Immediate 1200; Direct 4500; Indirect 999; Register 1500; Register indirect 6000; Relative EA 8200 (its operand is not listed).

Part 2: the method behind these, taught from nothing

Learn by Doing: Memory, Buses and CPU Components

Topic Eight

Part B: every past question the examiner has set on this topic

Question One: 2020/2021, Q1(a)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2020/2021, Question 1(a), and in 1 other paper

THE COURSE TEXT, UNEDITED

- a. Minicomputers in the eighties had 18 address signals and 18-bit address bus. Answer the following questions: **3mks**
- i. What was the address space of these computers?
 - ii. What may have been the largest possible memory of these computers in bytes if the memory location is 1 Byte?
 - iii) How long should the program counter (PC) of these computers be?

BREAK IT DOWN FIRST

before you write a word

What is this question actually asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given 18 address signals, which is the same thing as an 18-bit address bus, and (from the wording of part ii only) one byte per memory location.

Asked three separate quantities. (i) the *address space*, which is a count of locations, not a size in bytes. (ii) the largest memory *in bytes*. (iii) "How long should the program counter (PC) of these computers be?", meaning its width in bits.

Marks 3mks for all three sub-items together, so one mark each. That is one line of working per item, not an essay.

Taught in this unit, Module Two Unit 1.

Trap (i) and (ii) look like the same question and are not. The address space is a count of locations; the memory size is that count times the bytes in a location. They agree here only because the paper quietly sets a location to 1 byte, inside part (ii).

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Answer **2020/2021 Q1(a)** in full in your book, then open this and mark yourself against it.

GIVEN $k = 18$ address lines $\cdot$ 1 byte per location

FIND (i) the address space, (ii) the maximum memory in bytes, (iii) the width of the PC

FORMULA address space = $2^k \cdot$ capacity = $2^k \times$ bytes per location $\cdot$ PC width = k

WHAT EACH SYMBOL MEANS

k = the number of address lines, that is the address-bus width in bits $\cdot$ 2^k = how many distinct locations those lines can name $\cdot$ the **PC** must be able to hold any address, so it is exactly k bits wide

1. Address space = $2^{\text{number of lines}} = 2^{18} = 262\,144$ locations. $2^{18} = 2^{10} \times 2^8 = 1024 \times 256$.

2. At one byte per location that is 262 144 bytes = 256 KB.

3. The PC must hold any address, so it is as wide as the address bus: 18 bits.

(i) $2^{18} = 262\,144$ locations. (ii) 262 144 bytes = 256 KB. (iii) PC length = 18 bits.

Now redo it: address space is always 2 to the power of the number of address lines, and the PC is always as wide as an address.

Question Two: 2020/2021, Q5(a)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2020/2021, Question 5(a), and in 1 other paper

THE COURSE TEXT, UNEDITED

- a. i) A processor can support a maximum memory of 4GB where the memory is word addressable (a word = 2 bytes). Determine the number of bits required to uniquely address the entire memory. **2mks**

BREAK IT DOWN FIRST

before you write a word

What is this question actually asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given a maximum memory of 4GB, word addressable, with a word of 2 bytes.

Asked "the number of bits required to *uniquely* address the entire memory". The word *uniquely* is the instruction to count addressable units, and the addressable unit here is the word.

Marks 2mks, so two steps: turn the capacity into a count of words, then take the base 2 logarithm.

Taught in this unit, Module Two Unit 1.

Trap answering 32. That is the count of *bytes*. Word addressable means each address names a 2-byte word, so divide before you take the log.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Answer **2020/2021 Q5(a)** in full in your book, then open this and mark yourself against it.

GIVEN memory = 4 GB · word size = 2 bytes · addressing = word-addressable (each address names a word, not a byte)

FIND k, the number of address bits needed

FORMULA $N = \text{total bytes} / \text{bytes per addressable unit}$, then $k = \log_2 N$

WHAT EACH SYMBOL MEANS

N = the number of addressable units, here *words* not bytes · **k** = address bits · the phrase **word-addressable** is the whole question: divide the byte total by the word size *before* taking the log

1. $4 \text{ GB} = 4 \times 2^{30} \text{ bytes} = 2^{32} \text{ bytes}$.

2. Word-addressable means each address names a word, not a byte. Number of words = $2^{32} \text{ bytes} \div 2 \text{ bytes} = 2^{31} \text{ words}$.

3. To address 2^{31} words needs **31** bits (because 2^{31} words need 31 address lines).

$4 \text{ GB} \div 2\text{-byte word} = 2^{31} \text{ words}$, so **31 address bits** are needed.

Now redo it: the trap is word-addressable versus byte- addressable. Divide the total bytes by the bytes per addressable unit first, then take the log base 2.

Question Three: 2020/2021, Q5(b)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2020/2021, Question 5(b)

THE COURSE TEXT, UNEDITED

b. i) List four (4) differences between DRAM and SRAM?

4mks

BREAK IT DOWN FIRST

before you write a word

What is this question actually asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given nothing. This is straight recall, and it is the cheapest mark on the paper if you have the table memorised.

Asked "List", so a list. Each line must be a *difference*, which means both sides named on one property.

Marks 4mks for four differences, one mark each. A fifth earns nothing, a third loses one.

Taught in this unit, the Teach box directly above.

Trap writing four statements about DRAM alone. A difference needs DRAM and SRAM contrasted in the same line, so set it out as pairs.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Answer **2020/2021 Q5(b)** in full in your book, then open this and mark yourself against it.

Dynamic RAM (DRAM)	Static RAM (SRAM)
Stores a bit as charge on a capacitor	Stores a bit in a transistor latch
Must be refreshed periodically	Needs no refresh while powered
Slower access	Faster access
Higher density, cheaper per bit	Lower density, dearer per bit
Used for main memory	Used for cache memory

Now redo it: the four you must have are refresh, speed, density and cost. The memory hook: **DRAM** is **D**ense, **D**ynamic (needs refresh) and cheap; **SRAM** is **S**tatic (no refresh), **S**peedy and costly.

Part 2: the method behind these, taught from nothing

Learn by Doing: Performance, RISC and CISC

Topic Nine

Part B: every past question the examiner has set on this topic

Question One: 2025/2026, Q1(d)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 1(d)

THE COURSE TEXT, UNEDITED

d. A processor executes one million(1M) instructions distributed as follows.

Addressing Mode	Percentage	Cycles
Register	40%	1
Immediate	20%	1
Direct	15%	2
Indirect	10%	4
Indexed	10%	3
Relative	5%	2

- i) Compute the average CPI 2mks
- ii) Determine total clock cycles. 1mk
- iii) If indirect addressing is optimized to 2 cycles, compute the new CPI. 1mk
- iv) Evaluate the performance improvement. 2mks

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

GIVEN 1,000,000 instructions ; per-mode share and cycles as tabled

FIND average CPI, total cycles, new CPI after optimising indirect, improvement

FORMULA $CPI = \sum(\text{fraction} \times \text{cycles}) \cdot \text{total} = IC \times CPI \cdot \text{improvement} = (\text{old} - \text{new})/\text{old}$

$$\begin{aligned}\text{Avg CPI} &= 0.40 \times 1 + 0.20 \times 1 + 0.15 \times 2 + 0.10 \times 4 + 0.10 \times 3 + 0.05 \times 2 \\ &= 0.40 + 0.20 + 0.30 + 0.40 + 0.30 + 0.10 \\ &= 1.70\end{aligned}$$

$$\text{Total cycles} = 1,000,000 \times 1.70 = 1,700,000$$

$$\begin{aligned}\text{Indirect 4} \rightarrow 2: \text{ new CPI} &= 1.70 - 0.10 \times 4 + 0.10 \times 2 \\ &= 1.70 - 0.20 = 1.50\end{aligned}$$

$$\text{Improvement} = (1.70 - 1.50) / 1.70 = 0.20 / 1.70 = 11.76\%$$

CPI 1.70; total cycles 1,700,000; new CPI 1.50; improvement 11.76%.

Question Two: 2025/2026, Q3(c)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 3(c), and in 1 other paper

THE COURSE TEXT, UNEDITED

- c. i) State Amdahl's law' **2mks**
ii) Assume in hypothetical system cache is 10 times faster than main memory and CPU accesses the cache 85% of the time. Using Amdahl's law, calculate the overall speed up of the system.

2mks

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

GIVEN cache 10 times faster than memory ; used 85% of the time

FIND state Amdahl's law, then the overall speedup

FORMULA $S = 1 / ((1-f) + f/s)$, with f the sped-up fraction and s its speedup

Amdahl's law states that the overall speedup obtainable by improving one part of a system is limited by the fraction of the total time that part is actually used: even if that part is made infinitely fast, the speedup can never exceed $1/(1-f)$, where f is the fraction of the time that is sped up.

$$\begin{aligned} f &= 0.85 \text{ (cache hits), } s = 10 \\ S &= 1 / ((1 - 0.85) + 0.85/10) \\ &= 1 / (0.15 + 0.085) \\ &= 1 / 0.235 \\ &= 4.26 \end{aligned}$$

$S = 4.26$, far below 10, because the 15% of slow accesses dominate.

Question Three: 2024/2025, Q4(a)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question 4(a), and in 2 other papers

THE COURSE TEXT, UNEDITED

- a. i) Compare Von Neumann Architecture and Harvard Architecture. Include diagram for each. **3mks** ii) Mention any four (4) clear differences between RISC and CISC. Give two (2) examples of processors in each category. **4mks**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN four architecture names and nothing numeric.

ASKED *compare* two architectures with diagrams, then *mention* four differences and *give* four example processors. Three verbs, and the diagrams are compulsory, not decoration.

MARKS $3 + 4 = 7$. In part (ii) that is four differences plus four processors for 4 marks, so half a mark a line: write in a two column table, not prose.

TAUGHT IN Module One Unit 1 for the architectures, Module Four Unit 1 for RISC and CISC.

TRAP forgetting the diagrams. They are explicitly asked for and carry their own marks, and a compare answered as prose alone cannot score full marks however good the prose is. The second trap is naming processors nobody can check: ARM and MIPS against x86 and 68000 are safe.

(i) Von Neumann uses **one** memory and **one** bus for instructions and data, so it is simpler and cheaper but the shared bus carries only one at a time. Harvard uses **separate** memories and buses, so an instruction fetch and a data access happen simultaneously: faster, but more hardware and less flexible. Draw both diagrams; they carry marks of their own.

Question Four: 2023/2024, Q1(b)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2023/2024, Question 1(b), and in 1 other paper

THE COURSE TEXT, UNEDITED

- b. i) State Amdahl's law' **2mks**
ii) Assume in hypothetical system cache is 10 times faster than main memory and CPU accesses the cache 85% of the time. Using Amdahl's law, calculate the overall speed up of the system. **2mks**

BREAK IT DOWN FIRST

before you write a word

What is this question actually asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given a cache 10 times faster than main memory, and a CPU that finds what it needs in the cache 85% of the time.

Asked two things. First "State" the law, which is a statement in words plus the formula, not the formula alone. Then "calculate the overall speed up of the system".

Marks 2mks (2½mks in 23/24) for the statement and 2mks for the number, so the words are worth as much as the arithmetic. Write the law out.

Taught in this unit, the Must memorise box directly above.

Trap reading "10 times faster" as the answer. That is s , and it applies only to the fraction $f = 0.85$. The 15% still going to main memory at full cost is what holds the whole machine down to about 4.26.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Answer **2023/2024 Q1(b)** in full in your book, then open this and mark yourself against it.

GIVEN $f = 0.85$ (the 85% of accesses served by cache) · $s = 10$ (cache is ten times faster than main memory)

FIND the overall speedup of the machine

FORMULA $\text{speedup} = 1 / ((1 - f) + f/s)$

WHAT EACH SYMBOL MEANS

f = the fraction of the work the improvement touches, here the cache hits · **s** = how many times faster that fraction runs · **(1 - f)** = the 15% still going to main memory at full cost, which is what caps the answer well below 10

1. The sped-up fraction is $f = 0.85$ (the cache accesses), and the speedup factor is $s = 10$.

2. $\text{Speedup} = 1 / ((1 - 0.85) + 0.85 / 10) = 1 / (0.15 + 0.085)$.

3. $= 1 / 0.235 = \mathbf{4.26}$.

$\text{Speedup} = 1 / (0.15 + 0.085) = 1 / 0.235 \approx \mathbf{4.26 \text{ times}}$.

Now redo it: notice the answer is far below 10, because the 15% of slow accesses dominate. That is exactly the point Amdahl's law makes.

Question Five: 2023/2024, Q5(c)

THE COURSE TEXT, UNEDITED

c. Mention any 3 clear differences between RISC and CISC. Give two examples of processors in each category. **3mks**

BREAK IT DOWN FIRST

before you write a word

What is this question actually asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given nothing. Pure recall, and the fastest marks on any of these three papers.

Asked a fixed number of "clear differences", then two example processors of each family. Both halves are marked and the examples are the half students forget.

Marks 4mks in 20/21 and 24/25, 3mks in 23/24. The wording tells you how many differences buy them.

Taught in this unit, the comparison table directly above.

Trap the word "clear". A difference has to contrast both sides on one property, so write each as a pair rather than as two separate descriptions.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Answer **2023/2024 Q5(c)** in full in your book, then open this and mark yourself against it.

Four differences (any four): RISC has few simple fixed-length one-cycle instructions and only load/store touch memory, with many registers and simple pipelined hardware; CISC has many complex variable-length multi-cycle instructions, many of which access memory directly, with fewer registers and complex hardware. Examples: RISC, ARM and MIPS (also SPARC, PowerPC); CISC, Intel x86 (8086, Pentium) and the Motorola 68000.

Now redo it: pick your four contrasts (instruction count, length, cycles, memory access) and memorise two examples each: ARM/MIPS for RISC, x86/68000 for CISC.

Question 6: 2020/2021, Q4(b)

THE COURSE TEXT, UNEDITED

b. i) State Four (4) clear differences between RISC and CISC. Give two examples of processors in each category. **4mks**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Answer **2020/2021 Q4(b)** in full in your book, then open this and mark yourself against it.

Four clear differences between RISC and CISC. This year asks for four and two example processors in each family; the 2023/2024 paper asks for three of the same differences, so one table answers both.

	RISC	CISC
Instruction set	Small and simple, each instruction doing one thing	Large and complex, single instructions doing multi step work
Instruction length	FIXED length, which makes decoding and pipelining simple	VARIABLE length, so decoding is harder and pipelining awkward
Memory access	LOAD and STORE only: arithmetic works on registers	Instructions may operate directly on memory operands
Control unit	Hardwired, and typically one instruction per cycle	Microcoded, and many cycles per instruction
Registers	Many general purpose registers	Fewer registers, with more addressing modes instead
Where the work goes	Into the compiler, which emits more instructions	Into the hardware, which does more per instruction

Two example processors in each category, which the question asks for and which most answers forget:

- **RISC:** ARM and MIPS. SPARC and PowerPC also earn the mark.
- **CISC:** the Intel x86 family and the Motorola 68000. The DEC VAX also earns it.

The closing sentence worth a mark: the two have converged, because modern x86 processors decode their complex instructions into simple internal operations, so the difference now lives in the instruction set the programmer sees rather than in how the chip is built.

Part 2: the method behind these, taught from nothing

Learn by Doing: Von Neumann, Harvard and Image Data

Topic Ten

Part B: every past question the examiner has set on this topic

Question One: 2025/2026, Q4(c)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 4(c), and in 1 other paper

THE COURSE TEXT, UNEDITED

c. Calculate the memory requirement of an image whose dimension is 17.12 inches by 9.45 inches at resolution 250 dpi. Express your answer in Megabyte

3mks

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN 17.12 in by 9.45 in at 250 dpi ; 1 bit per pixel
FIND the memory in megabytes

FORMULA pixels = inches × dpi (both ways) ; bytes = pixels × bits / 8 ; MB = bytes / 1024 / 1024

pixels across = 17.12 × 250 = 4280
pixels down = 9.45 × 250 = 2362.5
total pixels = 4280 × 2362.5 = 10,111,500
bytes = 10,111,500 / 8 = 1,263,937.5
MB = 1,263,937.5 / 1024 / 1024 = 1.21 MB

10,111,500 pixels × 1 bit = 1,263,937.5 bytes ≈ 1.21 MB. State the megabyte convention you used.

Question Two: 2024/2025, Q3(a)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question 3(a)

THE COURSE TEXT, UNEDITED

a. i) What do you understand by Von Neumann Concept? **1mk** ii) Mention any 2 significant aspects of Von Neumann Architecture. **2mks** iii) Describe the Von Neumann architecture bottleneck. **2mks**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN nothing but the name of the architecture. This is a recall part from end to end.

ASKED three different verbs on the same topic: what you *understand* by the concept, *mention* two aspects, and *describe* the bottleneck. Mention wants a list, describe wants a mechanism.

MARKS 1 + 2 + 2 = 5. The single mark on (i) tells you one sentence is enough there; the two marks on (iii) tell you a bare phrase is not.

TAUGHT IN Module One Unit 1, Architecture versus Organization.

TRAP describing the bottleneck as "the CPU is faster than memory". That is a different problem. The bottleneck named here is the single shared path that instructions and data must both travel, so no instruction fetch and data access can happen in the same instant.

The Von Neumann concept: a computer stores its program (instructions) and its data together in the **same** memory, and executes instructions one after another by fetching them from that memory. Two aspects: (a) a single shared memory holds both instructions and data; (b) instructions execute sequentially, one at a time, under the stored-program idea. The bottleneck: because one bus is shared between instructions and data, the CPU cannot fetch an instruction and access data at the same instant, so that single path limits overall speed however fast the CPU is.

Question Three: 2024/2025, Q3(b)(ii)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question 3(b)(ii), and in 1 other paper

THE COURSE TEXT, UNEDITED

ii) Calculate the memory requirement of an image whose dimension is 17.12 inches by 9.45 inches at resolution 250 dpi. Express your answer in Megabyte. **3mks**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN 17.12 inches by 9.45 inches at 250 dots per inch. No colour depth, which is the number you need and are not given.

ASKED *calculate* the memory requirement and *express* it in megabytes. Two instructions, and the second one is a unit conversion that carries its own mark.

MARKS 3. One for the pixel count, one for the conversion to bytes, one for reaching megabytes.

TAUGHT IN Module One Unit 2, Data Representation.

TRAP the silent choice of 1 MB. Dividing by 1024 twice gives about 1.21 MB; dividing by one million gives about 1.26 MB. Both are defensible, neither is safe unstated, so name your convention on the page.

GIVEN 17.12 in × 9.45 in · 250 dpi · 1 bit per pixel (black and white, not stated so assumed)

FIND the memory in MB

FORMULA bits = (w×dpi)(h×dpi)×bpp, then ÷8, ÷1024, ÷1024

$4280 \times 2362.5 = 10\,111\,500$ pixels × 1 bit = 10 111 500 bits = 1 263 937.5 bytes ≈ **1.21 MB** (dividing by 1024 twice). State your convention: dividing by one million instead gives about 1.26 MB.

Part 2: the method behind these, taught from nothing

Learn by Doing: Cache Mapping Functions

Topic Eleven

Part B: every past question the examiner has set on this topic

Question One: 2025/2026, Q2(a)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 2(a), and in 3 other papers

THE COURSE TEXT, UNEDITED

a. Assume a computer has 32-bit addresses. Each block stores 16 words. A direct-mapped cache has 256 blocks. In which block (line) of the cache (decimal value) would you look for each of the following addresses? Addresses are given in hexadecimal for convenience. **4mks**

i. CA2DFC12

ii. F24F25FF

- iii. 24BBABCD
- iv. C1D9EEF2

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

GIVEN 32-bit address ; 16 words per block ; direct-mapped, 256 blocks

FIND the cache line each hex address maps to, in decimal

FORMULA $\text{offset} = \log_2(\text{words/block})$; $\text{index} = \log_2(\text{lines})$; the index field IS the line

$\text{offset} = \log_2(16) = 4 \text{ bits}$ (one hex digit)

$\text{index} = \log_2(256) = 8 \text{ bits}$ (two hex digits)

So: drop the last hex digit (offset), read the next two (the line):

CA2DFC1|2 -> C1 = 193

F24F25F|F -> 5F = 95

24BBABC|D -> BC = 188

C1D9EEF|2 -> EF = 239

Lines: CA2DFC12 → 193, F24F25FF → 95, 24BBABCD → 188, C1D9EEF2 → 239.

Question Two: 2025/2026, Q2(c)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 2(c), and in 3 other papers

THE COURSE TEXT, UNEDITED

c. A processor has the following configuration: Main memory is 1 GB; Cache is 512 KB; Block size is 128 bytes and it is a 4-way Set Associative cache.

i) Determine the total number of cache blocks. 1mk

ii) Determine the number of sets. 1mk

- iii) Construct the complete address format. **1mk**
- iv) If the hit rate is 96%, cache access time is 2 ns, and main memory access time is 80 ns, calculate the Average Memory Access Time (AMAT). **2mks**
- v) Discuss how increasing associativity beyond four ways might affect performance. **2mks**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

GIVEN main memory 1 GB ; cache 512 KB ; block 128 B ; 4-way ; $h = 0.96$, $T_c = 2$ ns, $T_m = 80$ ns

FIND blocks, sets, the address format, the AMAT

FORMULA blocks = cache/block ; sets = blocks/ways ; tag = addr – set – offset ; $AMAT = T_c + (1-h) \times T_m$

Blocks = $512 \text{ KB} / 128 \text{ B} = 2^{19} / 2^7 = 4096$
 Sets = $4096 / 4 = 1024$
 Address = 1 GB = 2^{30} -> 30 bits
 offset = $\log_2(128) = 7$
 set = $\log_2(1024) = 10$
 tag = $30 - 10 - 7 = 13$
 $AMAT = T_c + (1-h)T_m = 2 + 0.04 \times 80 = 2 + 3.2 = 5.2 \text{ ns}$

4096 blocks; 1024 sets; address tag 13 / set 10 / offset 7; AMAT 5.2 ns. (v) More ways cuts conflict misses but needs more tag comparators, so beyond a point it adds cost and delay for little gain.

Question Three: 2025/2026, Q3(a)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 3(a), and in 3 other papers

THE COURSE TEXT, UNEDITED

- a. A computer system has the following characteristics:
- Main memory = 64 MB
 - Cache memory = 128 KB
 - Block size = 64 bytes
- i) Determine the number of cache lines **1½mks**
- ii) For Direct Mapping, determine: Number of index bits; Number of tag bits; Number of offset bits **1½mks**
- iii) For a 4-way Set Associative Cache, determine: Number of sets; Number of set index bits; Number of tag bits **1½mks**
- iv) Explain why fully associative mapping eliminates conflict misses but introduces another performance challenge. **1½mks**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN main memory 64 MB ; cache 128 KB ; block 64 B

FIND lines ; direct fields ; 4-way fields ; the fully-associative trade-off

FORMULA lines = cache/block ; offset = $\log_2(\text{block})$; tag = addr – index(or set) – offset

Lines = $128 \text{ KB} / 64 \text{ B} = 2^{17} / 2^6 = 2^{11} = 2048$

Address = 64 MB = 2^{26} -> 26 bits ; offset = $\log_2(64) = 6$

Direct: index = $\log_2(2048) = 11$; tag = $26 - 11 - 6 = 9$

4-way : sets = $2048/4 = 512$; set = $\log_2(512) = 9$; tag = $26 - 9 - 6 = 11$

2048 lines. Direct: index 11, tag 9, offset 6. 4-way: set 9, tag 11, offset 6. (iv) Fully associative lets any block go in any line, so no conflict misses, but every line's tag must be searched, which is slow and costly.

Question Four: 2025/2026, Q5(c)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 5(c), and in 2 other papers

THE COURSE TEXT, UNEDITED

- c. Explain the following terms **3mks**
- i. Locality of reference
- ii. Temporal locality

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

i) Locality of reference is defined as the tendency of a program to keep accessing the same small set of memory locations over a short period, a consequence of loops and subroutines; it is why a small fast cache can satisfy most accesses.

ii) Temporal (time) locality is defined as: if a memory location is accessed, it is very likely to be accessed **again soon** (the same instruction or variable reused). Its partner, **spatial (space) locality**, is defined as: if a location is accessed, the **nearby** addresses are very likely to be accessed next (as when stepping through an array).

Question Five: 2024/2025, Q1(b)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question 1(b), and in 1 other paper

THE COURSE TEXT, UNEDITED

- b. Determine the average memory access time for a machine with a cache hit rate of 80% and cache access time of 5 ns and main memory access time of 100 ns when: i. Simultaneous access memory organization is used. **2mks** ii. Hierarchical access

memory organization is used. **2mks** iii. Draw the diagrams representing the two scenarios **3mks**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN a cache hit rate of 80%, a cache access time of 5 ns and a main memory access time of 100 ns. Nothing else, so the miss rate 20% is yours to derive.

ASKED *determine* the average access time twice, once under each organisation, and then *draw* both. Three separate deliverables under one part.

MARKS $2 + 2 + 3 = 7$. The 3 sits on the diagrams, which is the largest single share, so a part answered with numbers only throws away nearly half of it.

TAUGHT IN Module Three Unit 3, Cache Memory Design.

TRAP using one formula for both. Simultaneous access starts cache and main memory together, so the cache time is not paid twice; hierarchical pays the cache time on every access and the memory time again on a miss. The answers differ by 1 ns, and a copied formula scores zero on one of the two.

GIVEN $h = 0.80 \cdot T_c = 5 \text{ ns} \cdot T_m = 100 \text{ ns}$

FIND AMAT under both organisations

FORMULA simultaneous = $h \times T_c + (1-h) \times T_m$ · hierarchical = $T_c + (1-h) \times T_m$

Simultaneous = $0.8(5) + 0.2(100) = 4 + 20 = \mathbf{24 \text{ ns}}$. Hierarchical = $5 + 0.2(100) = 5 + 20 = \mathbf{25 \text{ ns}}$.

Diagrams: simultaneous shows the CPU addressing cache and main memory in parallel; hierarchical shows the CPU reaching main memory only through the cache. Hierarchical is 1 ns slower here because the cache time is paid on every access, hit or miss.

Question 6: 2024/2025, Q1(c)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question 1(c), and in 3 other papers

THE COURSE TEXT, UNEDITED

c. Assume a computer has 24-bit addresses. Each block stores 16 words. A direct-mapped cache has 256 blocks. In which block (line) of the cache would you look for each of the following addresses? (Addresses are given in hexadecimal for convenience). **6mks** a. 1A2BC0 b. FFFF00 c. 123456 d. C109D5

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN a 24-bit address, 16 words per block, a direct-mapped cache of 256 blocks, and four addresses written in hexadecimal.

ASKED in which block (line) each address lands. One number per address, four numbers in all.

MARKS 6 for four addresses, so roughly $1\frac{1}{2}$ each with the remainder for showing the field split once. Do the split once, then apply it four times.

TAUGHT IN Module Three Unit 3, Cache Memory Design.

TRAP counting bits when you could count hex digits. 4 offset bits are exactly one hex digit and 8 index bits are exactly two, so the line is the two digits before the last. Also give the answer in decimal and say so, since the paper leaves the base of the answer open.

GIVEN address = 24 bits · block = 16 words · cache = 256 blocks · mapping = direct

FIND the cache block (line) for each address

FORMULA $\text{offset} = \log_2 16 = 4 \text{ bits}$ · $\text{index} = \log_2 256 = 8 \text{ bits}$ · $\text{tag} = 24 - 8 - 4 = 12 \text{ bits}$ · the index field is the line number

READING IT OFF THE HEX

4 offset bits are exactly **one hex digit**, and 8 index bits are exactly **two**. So in a 6-digit address, drop the last digit and the next two digits are the line.

Address	Offset (last digit)	Line (next two digits)	Line in decimal
1A2BC0	0	BC	188
FFFF00	0	F0	240
123456	6	45	69
C109D5	5	9D	157

Blocks: 1A2BC0 → **188**, FFFF00 → **240**, 123456 → **69**, C109D5 → **157**. Address format: tag 12, line 8, offset 4.

Question 7: 2024/2025, Q2(a)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question 2(a)

THE COURSE TEXT, UNEDITED

- a. i) Describe how cache memory improves the performance of a computer system.
- 2mks** ii) Using the specifications below, design a cache system for each of Direct, Fully Associative and 2-way Set Associative Mapping. iii) State one advantage and one disadvantage of each method. **8mks** specifications: - Cache size: 1 KB = 1024 bytes - Block size: 16 bytes - Main memory size: 64 KB = 65,536 bytes - Address size: 16 bits

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN cache 1 KB, block 16 bytes, main memory 64 KB, address 16 bits, and three mapping schemes to apply them to.

ASKED three verbs. *Describe* how cache improves performance, *design* the cache under each of three mappings, and *state* one advantage and one disadvantage of each. Nine separate statements in the last part alone.

MARKS 2 + 8 = 10, half of Question Two. The design and the trade offs share 8, so budget roughly 5 for the three address formats and 3 for the six verdicts.

TAUGHT IN Module Three Unit 3, Cache Memory Design.

TRAP recomputing the line count for each mapping. It is 1 KB / 16 B = 64 lines under all three; only the way those lines are addressed changes. The other trap is answering "design" with prose: the mark is for a tag, index and offset split in bits.

GIVEN cache = 1 KB · block = 16 bytes · main memory = 64 KB · address = 16 bits

FIND the address format under each of the three mappings, plus one advantage and one disadvantage each

FORMULA lines = cache / block · offset = $\log_2(\text{block})$ · sets = lines / k · tag = address – index (or set) – offset · tag directory = lines × tag bits

WHAT EACH SYMBOL MEANS

lines = 1 KB / 16 B = **64**, the same under all three mappings · **offset** = $\log_2 16 = 4$ bits · **k** = the ways per set, 2 for the 2-way case · 64 KB = 2^{16} , which is why the address is 16 bits

Mapping	Tag	Index / set	Offset	Tag directory
Direct	6	6 (index, 64 lines)	4	$64 \times 6 = 384$ bits = 48 bytes
Fully associative	12	none	4	$64 \times 12 = 768$ bits = 96 bytes
2-way set assoc.	7	5 (set, 32 sets)	4	$64 \times 7 = 448$ bits = 56 bytes

Direct: simplest and fastest lookup (one line to check, one comparator), but two blocks mapping to the same line evict each other, causing conflict misses. **Fully associative:** no conflict misses, any block may sit anywhere, but every line's tag must be compared at once, which is expensive and slow, and the tag directory doubles to 96 bytes. **2-way:** the compromise, most conflict misses removed for only two comparators and a modest tag growth to 56 bytes.

Question 8: 2024/2025, Q2(b)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT	2024/2025, Question 2(b), and in 2 other papers

b. i) Explain the following terms: **4½mks** 1. Locality of reference 2. Temporal locality 3. Special Locality ii) What happens when there is a Cache Miss? **1½ mks**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN nothing numeric. This is a definitions part, four terms in all if you count the cache miss.

ASKED *explain*, not merely name, three terms, then say what *happens* on a cache miss. The second verb wants a sequence of events, not a definition.

MARKS $4\frac{1}{2} + 1\frac{1}{2} = 6$, so $1\frac{1}{2}$ per term in (i). That is two or three sentences each, with an example, not one line.

TAUGHT IN Module Three Unit 1, Computer Memory Overview.

TRAP correcting the paper. Answer the third term as spatial locality, but keep the examiner's numbering and do not waste a line pointing out the spelling. The other trap is defining temporal and spatial without first defining locality of reference, which is the term the other two are special cases of.

(i) Locality of reference is the tendency of a program to access a small, predictable set of memory locations during any short period. **Temporal:** a location used now is likely to be used again soon, so cache keeps recently used items. **Spatial:** locations near one just used are likely to be used soon, so cache fetches a whole block of neighbours at once. (ii) On a miss the word is not in cache, so the address goes onto the system bus, main memory returns the **whole block**

containing it, that block is loaded into a cache line (evicting one if necessary), and the required word is delivered to the CPU at the same time.

Question 9: 2023/2024, Q1(d)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2023/2024, Question 1(d)

THE COURSE TEXT, UNEDITED

- d. If an 8-way set association cache is made up of 32 bits words, 4 words per line and 4096 sets.
- i) Size of each line of the cache 1mk
 - ii) Size of each set of the cache 1mk
 - iii) Size of the cache itself 1mk

BREAK IT DOWN FIRST

before you write a word

What is this question actually asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given 8 ways to a set, a 32-bit word, 4 words to a line, and 4096 sets.

Asked three sizes, each written as a noun phrase, "Size of each line of the cache". A number with a unit is the whole of the answer.

Marks 1mk each. There is no working mark on offer, so give the number and one line showing where it came from.

Taught in this unit, Module Three Unit 3.

Trap 32 bits is 4 bytes. Answering in bits where the natural unit is bytes, or multiplying by 32 where you meant 4, is where this one is lost. Build up in order: line, then set, then cache.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Answer **2023/2024 Q1(d)** in full in your book, then open this and mark yourself against it.

GIVEN $k = 8$ (8-way) · word = 32 bits = 4 bytes · 4 words per line · 4096 sets

FIND the size of a line, of a set, and of the whole cache

FORMULA line = words per line × word size · set = $k \times$ line size · cache = sets × set size

WHAT EACH SYMBOL MEANS

k = 8, the lines per set · a **32-bit word** is 4 bytes, and mixing the two units is the classic slip here · build it up in order, line then set then cache, and the arithmetic stays trivial

1. Line size = 4 words × 32 bits = 4 × 4 bytes = **16 bytes**.
2. Set size = 8 lines × 16 bytes = **128 bytes**.
3. Cache size = 4096 sets × 128 bytes = 524 288 bytes = **512 KB**.

Line 16 bytes, set 128 bytes, cache 512 KB.

Question 10: 2023/2024, Q3(a)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2023/2024, Question 3(a), and in 1 other paper

THE COURSE TEXT, UNEDITED

- a. Determine the average memory access time for a machine with a cache hit rate of 80% and cache access time of 5 ns and main memory access time of 100 ns when.
- i. Simultaneous access memory organization is used. **2½mks**
 - ii. Hierarchical access memory organization is used. **2½mks**
 - iii. Draw the diagrams representing the two scenarios **3mks**

BREAK IT DOWN FIRST

before you write a word

What is this question actually asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given a hit rate of 80%, a cache access time of 5 ns, and a main memory access time of 100 ns.

Asked the average access time under two *named* organisations, then the diagrams. The two names, "Simultaneous" and "Hierarchical", are the whole question: they choose which of the two formulas applies.

Marks 2 + 2 + 3 in 24/25, and 2½ + 2½ + 3 in 23/24. Either way the diagrams are worth more than either calculation, so draw them.

Taught in this unit, Module Three Unit 3.

Trap the two formulas differ in one place only: whether the cache time is weighted by the hit rate or paid on every access. Simultaneous starts both lookups at once; hierarchical searches the cache first. Give both numbers and label which is which.

Answer **2023/2024 Q3(a)** in full in your book, then open this and mark yourself against it.

GIVEN $h = 0.80 \cdot T_c = 5 \text{ ns} \cdot T_m = 100 \text{ ns} \cdot$ both organisations required

FIND the AMAT under (i) simultaneous and (ii) hierarchical organisation

FORMULA simultaneous = $h \times T_c + (1-h) \times T_m$ · hierarchical = $T_c + (1-h) \times T_m$

WHAT EACH SYMBOL MEANS

h = hit ratio · **T_c** , **T_m** = cache and main-memory access times · the *only* difference between the two formulas is whether the cache time is weighted by **h** (simultaneous, both started at once) or paid in full every time (hierarchical, cache searched first)

1. Simultaneous: $\text{AMAT} = h \times T_c + (1-h) \times T_m = 0.8 \times 5 + 0.2 \times 100 = 4 + 20 = \mathbf{24 \text{ ns.}}$

2. Hierarchical: $\text{AMAT} = T_c + (1-h) \times T_m = 5 + 0.2 \times 100 = 5 + 20 = \mathbf{25 \text{ ns.}}$

Simultaneous 24 ns, hierarchical 25 ns. Hierarchical pays the cache time on every access (even misses), so it is slightly slower here.

Now redo it: the difference is only whether the cache time is always paid (hierarchical) or only on hits (simultaneous). Learn both formulas.

Question 11: 2023/2024, Q3(c)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2023/2024, Question 3(c), and in 2 other papers

THE COURSE TEXT, UNEDITED

- c. Explain the following **4½mks**
- i) Locality of reference
 - ii) Temporal locality
 - iii) Spatial Locality

BREAK IT DOWN FIRST

before you write a word

What is this question actually asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given nothing but the terms. This is a definition question, not a calculation.

Asked "Explain", which is neither "define" nor "list". An explanation names the idea, says why it holds, and says what the machine does about it.

Marks 3mks for two terms in 25/26 and 4½mks for three in the two older papers, so about 1½ marks a term. That is a sentence of definition plus a sentence of consequence, each.

Taught in this unit, the Teach box directly above.

Trap the misspelling. Do not lose time in the hall wondering whether "Special Locality" or "Spacial Locality" is a fourth concept you have never met. It is spatial locality both times.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Answer **2023/2024 Q3(c)** in full in your book, then open this and mark yourself against it.

Locality of reference is the tendency of a program to access a small, predictable set of memory locations during any short period. Temporal locality: a location accessed now is likely to be accessed again soon (so cache keeps recently used items). Spatial locality: locations near one just accessed are likely to be accessed soon (so cache fetches a whole block of neighbouring words at once).

Part 2: the method behind these, taught from nothing

Learn by Doing: Pipelining and Hazards

Topic Twelve

Part B: every past question the examiner has set on this topic

Question One: 2025/2026, Q1(b)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 1(b)

THE COURSE TEXT, UNEDITED

b. A computer manufacturer is evaluating whether to replace a non-pipelined processor with a five-stage pipelined processor. The processor executes the following instruction mix: Arithmetic – 40%, Memory Access – 35%, Branch – 20%, Floating Point – 5%. The Non-pipelined processor clock cycle is 25ns. Pipelined processor stage times are: IF – 5ns, ID – 4ns, EX – 6ns, MEM – 5ns and WB – 4ns. Furthermore, Pipeline register overhead is 1 ns. Branch instructions incur a two-cycle penalty, Floating-point instructions has three

cycles for EX stage, Memory accesses experience cache misses 10% of the time, adding four extra cycles. The program contains 20 million instructions.

i) Determine the pipelined clock cycle. 1mk

ii) Calculate the actual CPI. 2mks

iii) Determine the actual execution time. 1mk

iv) Compute the speedup over the non-pipelined processor. 1mk

v) Explain why the speedup differs from the theoretical maximum. 1mk

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

GIVEN mix arithmetic 40%, memory 35%, branch 20%, floating point 5% ; five stage times 5, 4, 6, 5, 4 ns ; register overhead 1 ns ; non-pipelined clock 25 ns ; IC = 20 000 000 ; branch adds a 2-cycle penalty, floating point adds 4 extra cycles, a memory access misses 10% of the time adding 4 more.

FIND pipelined clock, actual CPI, execution time, speedup, and why it is short of the max

FORMULA clock = slowest stage + overhead · actual CPI = $1 + \sum(\text{fraction} \times \text{extra cycles})$ · time = IC × CPI × T · speedup = time(non-pipe) / time(pipe)

The stem is dense and does not say whether each penalty is extra or total, so state the reading: base CPI 1; branch +2; memory +(2 + 0.10×4) = 2.4; floating point +4; arithmetic +0.

$$\begin{aligned}\text{Clock} &= \max(5, 4, 6, 5, 4) + \text{overhead} \\ &= 6 + 1 = 7 \text{ ns}\end{aligned}$$

$$\begin{aligned}\text{CPI} &= 1 && \text{(base, one per cycle)} \\ &+ 0.20 \times 2 && \text{(branch)} \\ &+ 0.35 \times 2.4 && \text{(memory: } 2 + 0.10 \times 4) \\ &+ 0.05 \times 4 && \text{(floating point)} \\ &+ 0.40 \times 0 && \text{(arithmetic)} \\ &= 1 + 0.40 + 0.84 + 0.20 \\ &= 2.44\end{aligned}$$

$$\begin{aligned}\text{Time} &= \text{IC} \times \text{CPI} \times T \\ &= 20,000,000 \times 2.44 \times 7 \text{ ns} = 0.3416 \text{ s}\end{aligned}$$

$$\text{Non-pipe} = 20,000,000 \times 25 \text{ ns} = 0.5 \text{ s}$$

$$\text{Speedup} = 0.5 / 0.3416 = 1.46$$

Clock 7 ns; CPI 2.44; time 0.3416 s; speedup 1.46. (v) It is far below the ideal 5 because the branch, floating-point and cache-miss penalties and the register overhead all raise the CPI above 1.

Question Two: 2025/2026, Q2(b)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 2(b), and in 1 other paper

THE COURSE TEXT, UNEDITED

b. Consider the following code segment executed on a classic five-stage RISC processor.

```
I1: LW   R1,0(R2)
I2: ADD  R3,R1,R4
I3: SUB  R5,R3,R6
I4: SW   R5,8(R2)
I5: BEQ  R5,R0,LABEL
I6: MUL  R7,R5,R8
```

Assume:

- forwarding is available
- branch resolved in EX stage
- load-use hazards require one stall
- multiplier occupies EX for three cycles
- separate instruction and data caches

i) Identify and explain every pipeline hazard. **5mks**

ii) Draw the pipeline timing diagram **2mks**

iii) Determine the total execution cycles **1mk**

iv) Suggest any two hardware improvements that would reduce stalls. **1mk**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

GIVEN six instructions on a classic 5-stage RISC pipeline ; forwarding on ; branch resolved in EX ;

load-use costs 1 stall ; MUL holds EX for 3 cycles ; separate caches

FIND the hazards, the timing diagram, the total cycles, two improvements

FORMULA ideal = $(k + n - 1)$ cycles, then add every stall: 1 per load-use, and $(EX \text{ cycles} - 1)$ per multi-cycle execute

Hazards

I1→I2 is a **load-use RAW** (R1 not ready until MEM, so I2 stalls one cycle). I2→I3, I3→I4, I3→I5, I3→I6 are ordinary RAW hazards **hidden by forwarding**. I6's **multiply holds EX for three cycles**, adding two extra at the end. Structural hazards are avoided by the separate caches.

Timing diagram (one stall, then the long multiply)

Cyc	1	2	3	4	5	6	7	8	9	10	11	12	13
I1 LW	IF	ID	EX	ME	WB								
I2 ADD		IF	ID	--	EX	ME	WB						
I3 SUB			IF	--	ID	EX	ME	WB					
I4 SW					IF	ID	EX	ME	WB				
I5 BEQ						IF	ID	EX	ME	WB			
I6 MUL							IF	ID	EX	EX	EX	ME	WB

One load-use stall (the -- in cycle 4) plus two extra multiply cycles give a total of **13 cycles** (a clean run of 6 would be $5 + 6 - 1 = 10$). Two improvements: add more forwarding paths, and pipeline or duplicate the multiplier.

Question Three: 2025/2026, Q3(b)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 3(b)

THE COURSE TEXT, UNEDITED

b. A 5-stage pipelined processor has instruction fetch (IF), Instruction decode (ID), Operand Fetch (OF), Execute (Ex) and Write Back (WB) stages. The IF, ID, OF and WB stages take 1 clock cycle each for any instruction. The PO stage take 1 cycle for ADD and SUB instructions, 3 clock cycles for MUL instruction and 6 clock cycles for DIV instruction respectively. Operand forwarding is used in the pipeline.

Draw the cycle diagram and determine the number of cycles needed to execute the following sequence of instructions:

- When Operand forwarding is used **4½mks**
- When Operand forwarding is not used **5½mks**

I1 MUL R2, R0, R1 ; R2 ← R0 X R1

```

I2      DIV R5, R3, R4 ; R5 ← R3/R4
I3      ADD R2, R5, R2 ; R2 ← R5 + R2
I4      SUB R5, R2, R6 ; R5 ← R2 - R6

```

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

GIVEN 5 stages IF, ID, OF, EX, WB, one cycle each except EX = 1 (ADD/SUB), 3 (MUL), 6 (DIV) ; four instructions ; do it with and without forwarding

FIND the cycle diagram and total cycles for each case

FORMULA no single formula once EX is multi-cycle: lay the schedule out cycle by cycle, a long EX blocks those behind it, then read the total off the last write-back

The paper prints "PO stage", struck through in pen to EX. Instructions: I1 MUL R2,R0,R1 ; I2 DIV R5,R3,R4 ; I3 ADD R2,R5,R2 (needs R5 from I2 and R2 from I1) ; I4 SUB R5,R2,R6 (needs R2 from I3). Read each cell as the stage that instruction is in that cycle; "--" is a stall.

(i) With operand forwarding

Instr 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

I1 MUL IF ID OF EX EX EX WB

I2 DIV IF ID OF -- -- EX EX EX EX EX WB

I3 ADD IF ID -- -- OF -- -- -- -- EX WB

I4 SUB IF -- -- ID -- -- -- -- OF EX WB

The DIV holds EX for six cycles (7 to 12) and blocks everything behind it. With forwarding, I3 takes R5 straight from I2's EX and runs at cycle 13, and I4 takes R2 from I3 and runs at 14, so the last write-back is at **cycle 15**.

(ii) Without operand forwarding

Instr 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

I1 MUL IF ID OF EX EX EX WB

I2 DIV IF ID OF -- -- EX EX EX EX EX WB

I3 ADD IF ID -- -- OF -- -- -- -- EX WB

I4 SUB IF -- -- ID -- -- -- -- -- -- -- -- OF EX WB

Without forwarding, I3 must wait for I2's **write-back** before it can read R5 (one cycle later, EX at 14), and I4 likewise waits for I3, so the last write-back slips to **cycle 17**.

Total = **15 cycles** with forwarding, **17 cycles** without. Forwarding saved 2 cycles here.

Question Four: 2025/2026, Q5(b)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question 5(b), and in 1 other paper

THE COURSE TEXT, UNEDITED

- b. Consider a pipeline having 4 phases with duration 60, 50, 90 and 80 ns. Given latch delay is 10 ns. Calculate-
- i. Pipeline cycle time **1mk**
 - ii. Non-pipeline execution time **1mk**
 - iii. Speed up ratio **1mk**
 - iv. Pipeline time for 500 tasks **1mk**
 - v. Sequential time for 500 tasks **1mk**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN 4 phases 60, 50, 90, 80 ns ; latch 10 ns ; 500 tasks

FIND cycle time, one-task time, speedup, and the times for 500 tasks

FORMULA cycle = slowest + latch ; pipe(n) = (k + n - 1) × cycle ; seq(n) = n × Σphases ; speedup = Σphases / cycle

Cycle = $\max(60, 50, 90, 80) + 10 = 90 + 10 = 100 \text{ ns}$
One task = $60 + 50 + 90 + 80 = 280 \text{ ns}$
Speedup = $280 / 100 = 2.8$
Pipe 500 = $(4 + 500 - 1) \times 100 = 503 \times 100 = 50,300 \text{ ns}$
Seq 500 = $500 \times 280 = 140,000 \text{ ns}$

Cycle 100 ns; one task 280 ns; speedup 2.8; 500 tasks pipelined 50,300 ns vs sequential 140,000 ns.

Question Five: 2024/2025, Q5(b)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question 5(b), and in 1 other paper

THE COURSE TEXT, UNEDITED

b. Consider a pipeline having 4 phases with duration 60, 50, 90 and 80 ns. Given latch delay is 10 ns. Calculate- i. Pipeline cycle time **1mk** ii. Non-pipeline execution time **1mk** iii. Speed up ratio **1mk** iv. Pipeline time for 500 tasks **1mk** v. Sequential time for 500 tasks **1mk** vi. Throughput **1mk**

BREAK IT DOWN FIRST

before you write a word

Before you write: what are you given, what is actually asked, and what do the marks tell you about the length of the answer?

ANSWER

Work it out first Read the question twice, list what you are given, and say in one line what the answer will look like: a number, a table, a diagram, or a list of points. Then open the answer and mark yourself against it.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Cover the answer and work this yourself first, from the question and method set out just above. Then reveal and check line by line.

GIVEN four stage durations, 60, 50, 90 and 80 ns, a latch delay of 10 ns, and 500 tasks.

ASKED six numbers, in the paper's order. Nothing here asks for explanation, so every mark is a figure with its unit.

MARKS 6, one per item. Answer all six even if one is guessed: an omitted line cannot score, a wrong line costs only itself.

TAUGHT IN Module Two Unit 3, Pipelining.

TRAP leaving the latch out of the cycle time. The cycle is the slowest stage *plus* the latch, $90 + 10 = 100$ ns, and every later item depends on it, so one omission poisons five answers. The second trap is the $k + n - 1$ formula: 500 tasks through 4 stages take 503 cycles, not 504.

GIVEN $k = 4$ stages · 60, 50, 90, 80 ns · latch = 10 ns · $n = 500$ tasks

FIND cycle time, one-task non-pipelined time, speedup, 500-task pipelined and sequential times, throughput

FORMULA cycle = slowest + latch · pipelined = $(k + n - 1) \times$ cycle · sequential = $n \times \Sigma$ stages · throughput = $n /$ pipelined time

Cycle = $90 + 10 = 100$ ns. Non-pipelined one task = $60 + 50 + 90 + 80 = 280$ ns. Ideal speedup = $280/100 = 2.8$. For 500 tasks: pipelined = $(4 + 499) \times 100 = 50\,300$ ns; sequential = $500 \times 280 = 140\,000$ ns. Throughput = $500 / 50\,300$ ns, which in the steady state is one task per cycle, that is $1/100$ ns = 10 million tasks per second.

Question 6: 2020/2021, Q1(b)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2020/2021, Question 1(b)

THE COURSE TEXT, UNEDITED

b. i) What is pipelining? **2mks**
ii) Calculate how long it will take to execute on a regular non-pipelined and pipelined architecture. There are 4 operations with 5 stages each: IF, ID, EX, M and WB; stage1= 8ns, stage2 = 9ns, stage3 = 11ns, stage4 = 13ns, stage5 =16ns. Provide the Gantt Charts for both pipeline and non-pipeline operations. Also calculate the Speedup of the Pipeline system **6mks**

BREAK IT DOWN FIRST

before you write a word

What is this question actually asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given four operations, five stages, five stage times, and no latch delay mentioned at all, so take the latch as zero and say that you have.

Asked three deliverables packed into one sentence: the two execution times, the two charts, and the speedup. A student who computes the numbers and draws nothing has left marks that were never about arithmetic.

Marks 2mks for the definition in (i) and 6mks for (ii), which is the largest single part of that question.

Taught in this unit, Module Two Unit 3. Part (i), the definition of pipelining, is the opening paragraph of this unit.

Trap the non-pipelined time is n times the *sum* of the stages, 4 times 57. The pipelined time uses only the *slowest* stage, 16 ns, as its clock. Mixing the two is the usual loss here.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Answer **2020/2021 Q1(b)** in full in your book, then open this and mark yourself against it.

GIVEN $k = 5$ stages · stage times = 8, 9, 11, 13, 16 ns · $n = 4$ operations · no latch delay stated, so take it as 0

FIND the non-pipelined time, the pipelined time, and the speedup

FORMULA cycle = slowest stage · non-pipelined = $n \times \Sigma(\text{stage times})$ · pipelined = $(k + n - 1) \times \text{cycle}$ · speedup = non-pipelined / pipelined

WHAT EACH SYMBOL MEANS

$k = 5$ stages · $n = 4$ operations · $\Sigma(\text{stage times}) = 8+9+11+13+16 = 57$ ns, the full journey one operation makes without a pipeline · **cycle** = 16 ns, the slowest stage, because every stage must be given that long

1. Non-pipelined: each operation passes through all stages, $8+9+11+13+16 = 57$ ns each, so $4 \times 57 = \mathbf{228 \text{ ns}}$.
2. Pipelined cycle time = slowest stage = 16 ns.
3. Pipelined time = $(k + n - 1) \times \text{cycle} = (5 + 4 - 1) \times 16 = 8 \times 16 = \mathbf{128 \text{ ns}}$.
4. Speedup = $228 / 128 = \mathbf{1.78}$.

Non-pipelined 228 ns, pipelined 128 ns, speedup ≈ 1.78 . The speedup is below 5 because only 4 tasks run, so the fill time still dominates.

Question 7: 2020/2021, Q1(c)

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2020/2021, Question 1(c), and in 1 other paper

THE COURSE TEXT, UNEDITED

- c. i) Describe the three types of data hazards. **1½mks**
- ii) What type of data hazards do the following instructions belong? **1½ mks**
1. MULTD F0, F2, F4
DIVD F10, F0, F6
 2. DIVD F10, F0, F6
ADDD F6, F8, F2
 3. MULTD F0, F2, F4
SD F8, 50 (R5)

BREAK IT DOWN FIRST

before you write a word

What is this question actually asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given for (i) nothing, it is recall. For (ii) three pairs of floating-point instructions, each pair to be classified.

Asked (i) "Describe the three types". The three names on their own are not a description: each needs the order of the two clashing accesses spelled out. (ii) classify each pair, which is a different skill from reciting the list.

Marks 1½mks for three types, half a mark each, so one short sentence apiece; then 1½ mks for three classifications, again half a mark each.

Taught in this unit, the Teach box directly above.

Trap classifying by instruction name instead of by register. The mnemonic tells you nothing; only the registers do. Write down what each instruction *writes* and what it *reads*, then look for the same name on both sides. That method is also the only thing that shows you pair 3 has no overlap at all.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Answer **2020/2021 Q1(c)** in full in your book, then open this and mark yourself against it.

(i)

Read after write (RAW), write after read (WAR), and write after write (WAW). RAW is a true dependency; WAR and WAW are name dependencies that arise when instructions are reordered.

(ii) Take each pair and list what it writes and what it reads. The register, not the mnemonic, decides the answer.

Pair	Writes	Reads	Register in common	Hazard
------	--------	-------	--------------------	--------

1. MULTD F0, F2, F4 DIVD F10, F0, F6	F0 F10	F2, F4 F0, F6	F0: written first, read second	RAW
2. DIVD F10, F0, F6 ADDD F6, F8, F2	F10 F6	F0, F6 F8, F2	F6: read first, written second	WAR
3. MULTD F0, F2, F4 SD F8, 50 (R5)	F0 memory at R5+50	F2, F4 F8, R5	none	none as printed

1 is **RAW**, a true dependency: DIVD needs the F0 that MULTD has not finished writing. 2 is **WAR**, an anti-dependency: ADDD would overwrite the F6 that DIVD still has to read. 3, **as the paper prints it, is no hazard at all**: the two instructions touch no register in common.

What to write in the hall. Answer pair 3 the way the page is printed, then add one line: "as printed these two share no register, so there is no data hazard; had the store referenced F0 it would be WAW, write after write." That earns the mark for the analysis and shows the examiner you read the registers rather than pattern matching on the position in the list.

Part 2: the method behind these, taught from nothing

Mock Paper One

Mock paper

Question One

MODEL ANSWER

Question One

Write your full answer to **Question One** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION ONE

(a) Architecture vs organization; Von Neumann vs Harvard (see M1 U1)

Architecture is the programmer-visible design (instruction set, registers, addressing modes): what the machine does. Organization is how that is built in hardware (control signals, buses, memory technology): how it does it. Von Neumann uses one memory and one bus for instructions and data (simple, but the bus is a bottleneck); Harvard uses separate memories and buses (simultaneous access, faster, costlier). Draw both diagrams.

(b) CPI (see M2 U2)

(i) $CPI = 0.45(1) + 0.25(4) + 0.15(2) + 0.15(3) = 0.45 + 1.00 + 0.30 + 0.45 = \mathbf{2.20}$. (ii) Total cycles = $2\,000\,000 \times 2.20 = \mathbf{4\,400\,000}$. (iii) With Load at 2 cycles: $CPI = 0.45 + 0.25(2) + 0.30 + 0.45 = \mathbf{1.70}$. (iv) Improvement = $(2.20 - 1.70) / 2.20 = \mathbf{22.73\%}$.

(c) 12-bit representations (see M1 U2)

150 = 0000 1001 0110 (positive, so identical in all four schemes). For -95: +95 = 0000 0101 1111.

Value	Unsigned	Sign-magnitude	One's complement	Two's complement
150	0000 1001 0110	0000 1001 0110	0000 1001 0110	0000 1001 0110
-95	not representable	1000 0101 1111	1111 1010 0000	1111 1010 0001

(d) IEEE-754 (see M1 U3)

(i) $39.5 = 100111.1 = 1.001111 \times 2^5$. Exponent $5 + 127 = 132 = 1000\ 0100$. Sign 0. Mantissa 001111 padded to 23 bits. Word = 0 10000100 001111000000000000000000 = **421E0000H**. (ii) $42480000H = 0100\ 0010\ 0100\ 1000\dots$: sign 0, exponent $1000\ 0100 = 132$ so $e = 5$, mantissa $1001\dots$, value = $1.1001 \times 2^5 = 110010 = \mathbf{50.0}$.

(e) Cache fields (see M3 U3)

Offset = $\log_2 32 = 5$ bits; index = $\log_2 1024 = 10$ bits; tag = $32 - 10 - 5 = \mathbf{17\ bits}$.

Question Two

MODEL ANSWER

Question Two

Write your full answer to **Question Two** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION TWO

(a) Pipeline timing (see M2 U3)

(i) Cycle time = slowest stage + latch = $25 + 5 = \mathbf{30\ ns}$. (ii) Non-pipeline one task = $20 + 15 + 25 + 20 + 10 = \mathbf{90\ ns}$. (iii) Pipeline for 1000 tasks = $(5 + 999) \times 30 = 1004 \times 30 = \mathbf{30\ 120\ ns}$. (iv) Sequential for 1000 = $1000 \times 90 = \mathbf{90\ 000\ ns}$. (v) Speedup = $90\ 000 / 30\ 120 = \mathbf{2.99}$.

(b) Hazards (see M2 U3, M4 U2)

Structural (two instructions need the same hardware: duplicate it), data (a value is needed before it is written back: use forwarding, and one stall for a load-use case), control (a branch changes the next instruction: resolve early or flush). Forwarding feeds a result straight from where it is produced; a load value is not ready until the memory stage, so the very next instruction still needs one stall.

(c) Why speedup < stages

The pipeline must first fill (the first task still takes k cycles), and hazards insert stalls, so the ideal ceiling of k is never quite reached, especially for few tasks.

Question Three

MODEL ANSWER

Question Three

Write your full answer to **Question Three** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION THREE

(a) Cache configuration (see M3 U3)

(i) Blocks = cache / block = 64 KB / 32 B = $2^{16} / 2^5 = \mathbf{2048 \text{ blocks}}$. (ii) Sets = 2048 / 4 = **512 sets**. (iii) Address = 256 MB = 2^{28} , so 28 bits. Offset = $\log_2 32 = 5$, set = $\log_2 512 = 9$, tag = $28 - 9 - 5 = \mathbf{14 \text{ bits}}$. Format: tag 14, set 9, offset 5.

(b) AMAT (see M3 U3)

Simultaneous = $0.9(3) + 0.1(60) = 2.7 + 6 = \mathbf{8.7 \text{ ns}}$. Hierarchical = $3 + 0.1(60) = 3 + 6 = \mathbf{9.0 \text{ ns}}$.

(c) Locality (see M3 U1)

Locality of reference is the clustering of accesses in a small region over any short time. Temporal: a location used now is likely used again soon (so cache keeps recently used data). Spatial: nearby locations are likely used soon (so cache fetches whole blocks of neighbours).

Question Four

MODEL ANSWER

Question Four

Write your full answer to **Question Four** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION FOUR

(a) $Y = (A + B) \times (C - D)$ in four formats (see M1 U4)

3-address	2-address	1-address	0-address
ADD T1,A,B SUB T2,C,D MUL Y,T1,T2	MOV T1,A ADD T1,B MOV T2,C SUB T2,D MUL T1,T2 MOV Y,T1	LOAD A ADD B STORE T1 LOAD C SUB D MUL T1 STORE Y	PUSH A PUSH B ADD PUSH C PUSH D SUB MUL POP Y

(b) Addressing modes (see M1 U4)

Mode	Effective address	Operand
Direct	3200	M[3200] = 1500
Indirect	M[3200] = 1500	M[1500] = 700
Register-indirect (R1)	R1 = 2500	M[2500] = 900
Relative	PC + 3200 = 7200	M[7200] = 350

(c) RISC vs CISC (see M4 U1)

RISC: few simple fixed-length one-cycle instructions, load-store, many registers (example ARM or MIPS). CISC: many complex variable-length multi-cycle instructions, direct memory access, fewer registers (example Intel x86 or Motorola 68000).

Question Five

MODEL ANSWER

Question Five

Write your full answer to **Question Five** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION FIVE

(a) Amdahl's law (see M2 U2)

Stating it (the question says "state", so a formula alone does not answer it): *the gain in overall performance from improving one part of a system is limited by the fraction of the total time that part accounts for.* If a fraction f of the execution time is made s times faster and the rest is unchanged, the overall speedup is $S = 1 / ((1 - f) + f/s)$; and as s grows without bound S tends to $1 / (1 - f)$, which is the ceiling no improvement to that part can pass.

Applying it. With $f = 0.90$ and $s = 8$: Speedup = $1 / (0.10 + 0.90 / 8) = 1 / (0.10 + 0.1125) = 1 / 0.2125 = \mathbf{4.71}$. Sanity check against the ceiling: $1 / (1 - 0.90) = 10$, and 4.71 is below it, as it must be.

(b) DRAM vs SRAM (see M3 U2)

DRAM stores a bit as capacitor charge, needs refresh, is slower, denser and cheaper, and is used for **main memory**. SRAM stores a bit in a transistor latch, needs no refresh, is faster, less dense and dearer, and is used for **cache**. Cache needs speed; main memory needs cheap capacity.

(c) Memory access methods (see M3 U1)

Sequential (read in order, example magnetic tape), direct (jump near then search, example disk), random (any location in equal time, example RAM), associative (located by content, example a cache tag store).

Mock Paper Two

Mock paper

Question One

MODEL ANSWER

Question One

Write your full answer to **Question One** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION ONE

(a) Von Neumann (see M1 U1)

(i) The Von Neumann concept: a stored-program computer keeps instructions and data in the same memory and executes instructions sequentially. Two aspects: a single shared memory for code and data; sequential fetch-execute of a stored program. (ii) The bottleneck: the single shared bus lets only instructions or data move at a time, so the CPU waits on memory. Reduce it with cache and with separate instruction and data caches (or wider buses).

(b) 16-bit two's complement (see M1 U2)

(i) Range = -2^{15} to $+(2^{15}-1) = -32768$ to 32767 ; 300, -200 and -5000 all fit.

Value	Binary (16-bit two's complement)	Hex
300	0000 0001 0010 1100	012C
-200	1111 1111 0011 1000	FF38
-5000	1110 1100 0111 1000	EC78

(iii) $300 + (-200)$: add the patterns $012C + FF38 = 1\ 0064$, discard the carry out of bit 16, leaving $0064H = \mathbf{100}$. Check: $300 - 200 = 100$.

(c) IEEE-754 (see M1 U3)

(i) $85.75 = 1010101.11 = 1.01010111 \times 2^6$. Exponent $6 + 127 = 133 = 1000\ 0101$. Mantissa 01010111 to 23 bits. Word = $0\ 10000101\ 01010111000000000000000 = \mathbf{42AB8000H}$. (ii) $C2960000H$: sign 1, exponent $1000\ 0101 = 133$ so $e = 6$, mantissa 00101110 , value = $-1.0010110 \times 2^6 = -1001011 = \mathbf{-75.0}$.

(d) Instruction formats and bytes (see M1 U4)

3-address (opcode 1, three addresses 2 each = 7 bytes each):

ADD T1, A, B ADD T2, C, D MUL T1, T1, T2 ADD Z, T1, E

4 instructions $\times 7 = \mathbf{28\ bytes}$. 0-address (postfix $A\ B + C\ D + \times E +$): PUSH A, PUSH B, ADD, PUSH C, PUSH D, ADD, MUL, PUSH E, ADD, POP Z = 10 instructions = 5 pushes $\times 3 + 1\ pop \times 3 + 4\ ops \times 1 = \mathbf{22\ bytes}$.

Question Two

MODEL ANSWER

Question Two

Write your full answer to **Question Two** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION TWO

(a) Cache fields and block lookup (see M3 U3)

(i) Offset = $\log_2 8 = 3$, index = $\log_2 512 = 9$, tag = $32 - 9 - 3 = \mathbf{20}$. (ii) Block = (address with the low 3 offset bits removed) mod 512, that is (address $\div 8$) mod 512. ABCD1234H gives block **70**; 5678FEDCH gives block **475**.

(b) The three mapping functions (see M3 U3)

Direct: one line per block (fast and simple, but conflict misses). Fully associative: a block may go anywhere (no conflict misses, but every tag must be searched, costly). Set associative: a block maps to one set of k lines (the compromise, fewer conflicts than direct at less cost than fully associative).

Question Three

MODEL ANSWER

Question Three

Write your full answer to **Question Three** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION THREE

(a) 10-bit representations (see M1 U2)

200 = 0011001000 (positive, same in all four). For -75: +75 = 0001001011.

Value	Unsigned	Sign-magnitude	One's complement	Two's complement
200	0011001000	0011001000	0011001000	0011001000
-75	not representable	1001001011	1110110100	1110110101

(b) End-around carry (see M1 U2)

$200 - 75 = 200 + (-75)$. 200 = 0011001000; -75 in one's complement = 1110110100. Add: 0011001000 + 1110110100 = 1 0001111100, a carry out of bit 10. End-around: drop it and add 1 to the low bit: 0001111100 + 1 = 0001111101 = **125**. Check: $200 - 75 = 125$.

(c) Image memory (see M1 U2)

Pixels = $(8 \times 200) \times (10 \times 200) = 1600 \times 2000 = 3\,200\,000$. At 1 bit per pixel that is 3 200 000 bits = 400 000 bytes = 400 000 / 1 048 576 = **0.381 MB** (about 0.38 MB).

Question Four

MODEL ANSWER

Question Four

Write your full answer to **Question Four** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION FOUR

(a) IEEE-754 reverse (see M1 U3)

43A00000H = 0100 0011 1010 0000...: sign 0, exponent 1000 0111 = 135 so $e = 8$, mantissa 0100000. Value = $1.0100000 \times 2^8 = 101000000 = \mathbf{320.0}$.

(b) Instruction categories (see M1 U4)

Data transfer (LOAD, STORE, MOVE, PUSH, POP), arithmetic and logic (ADD, SUB, MUL, AND, OR), control or test-and-branch (JUMP, BEQ, CALL, RETURN).

(c) Addressing modes (see M1 U4)

Immediate: the operand is the address field itself (a constant). Direct: the operand is $M[\text{address field}]$. Indirect: the field points to a location that holds the address, so the operand is $M[M[\text{address field}]]$. Register-indirect: a register holds the address, so the operand is $M[\text{register}]$.

Question Five

MODEL ANSWER

Question Five

Write your full answer to **Question Five** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION FIVE

(a) Pipeline timing (see M2 U3)

Cycle time = slowest stage + latch = $50 + 5 = \mathbf{55 \text{ ns}}$. Pipeline time for 200 tasks = $(4 + 199) \times 55 = 203 \times 55 = \mathbf{11\ 165 \text{ ns}}$. Sequential time = $200 \times (40 + 30 + 50 + 35) = 200 \times 155 = \mathbf{31\ 000 \text{ ns}}$. Speedup = $31\ 000 / 11\ 165 = \mathbf{2.78}$.

(b) Memory hierarchy (see M3 U1)

Registers, then cache (L1, L2, L3), then main memory (RAM), then secondary storage (disk, SSD), then tertiary (tape). Going down: bigger, slower and cheaper per byte. Going up: smaller, faster and dearer.

(c) AMAT (see M3 U3)

Hierarchical: $AMAT = T_{\text{cache}} + (1 - h) \times T_{\text{memory}} = 4 + 0.15 \times 70 = 4 + 10.5 = \mathbf{14.5 \text{ ns}}$.

Mock Paper Three

Mock paper

Question One

MODEL ANSWER

Question One

Write your full answer to **Question One** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION ONE

(a) Architecture versus organization (2 marks) [M1 U1]

Architecture is the set of attributes visible to the programmer (instruction set, registers, addressing modes): **what** the machine does. Organization is how those are realised in hardware (control signals, buses, memory technology): **how** it does it. Example: the architecture says a MULTIPLY instruction exists; the organization decides whether a dedicated multiplier or repeated addition carries it out.

(b) 178 and -92 in 11 bits (6 marks) [M1 U2]

GIVEN values 178 and -92 · n = 11 bits · four schemes

FIND each value in each scheme, plus the two's complement range

FORMULA sign-magnitude = sign bit + magnitude · one's complement = invert all n bits · two's complement = invert then add 1 · range = -2^{n-1} to $+(2^{n-1}-1)$

1. $178 = 128 + 32 + 16 + 2$, so in 11 bits $178 = 000\ 1011\ 0010$. Positive, so identical in all four schemes.
2. For -92, first write $+92 = 64 + 16 + 8 + 4 = 000\ 0101\ 1100$.
3. Sign-magnitude: set the sign bit, keep the magnitude: $100\ 0101\ 1100$.
4. One's complement: invert all 11 bits of +92: $111\ 1010\ 0011$.
5. Two's complement: add 1 to that: $111\ 1010\ 0100$.

Value	Unsigned	Sign-magnitude	One's complement	Two's complement
178	000 1011 0010	000 1011 0010	000 1011 0010	000 1011 0010
-92	not representable	100 0101 1100	111 1010 0011	111 1010 0100

11-bit two's complement range = -2^{10} to $+(2^{10}-1) = -1024$ to **1023**. Both 178 and -92 fall well inside it, so both are representable.

(c) 92.375 in IEEE-754 single precision (5 marks) [M1 U3]

GIVEN value = 92.375 · IEEE-754 single precision · bias 127

FIND the 32-bit pattern, in binary and hex

FORMULA normalise to $1.m \times 2^e$, then $E = e + 127$

1. Sign: positive, so **0**.
2. Binary: $92 = 1011100$ and $0.375 = 0.011$, so $92.375 = 1011100.011$.
3. Normalise: move the point 6 places left, 1.011100011×2^6 , so $e = 6$. This one terminates, so it is exact.
4. Exponent field: $6 + 127 = 133 = 1000\ 0101$.
5. Mantissa: 011100011 padded to 23 bits = $0111\ 0001\ 1000\ 0000\ 0000\ 000$.

92.375 = 0 10000101 01110001100000000000000 = 0100 0010 1011 1000 1100 0000 0000 0000 = 42B8C000H. Converting back gives 92.375 exactly, which is the check to write down.

(d) CPI over 2 500 000 instructions (6 marks) [M2 U2]

GIVEN IC = 2 500 000 · A 35% at 1, B 25% at 2, C 20% at 3, D 12% at 4, E 8% at 5 · for (iii), class D falls to 2 cycles

FIND average CPI, total cycles, new CPI, percentage improvement

FORMULA $CPI = \sum(\text{fraction}_i \times CPI_i) \cdot \text{cycles} = IC \times CPI \cdot \text{improvement} = (\text{old} - \text{new})/\text{old}$

1. $CPI = 0.35(1) + 0.25(2) + 0.20(3) + 0.12(4) + 0.08(5) = 0.35 + 0.50 + 0.60 + 0.48 + 0.40 = \mathbf{2.33}$.
2. Total cycles = 2 500 000 × 2.33 = **5 825 000**.
3. Class D from 4 to 2 cycles: $CPI = 2.33 - 0.12(4) + 0.12(2) = 2.33 - 0.48 + 0.24 = \mathbf{2.09}$.
4. Improvement = $(2.33 - 2.09) / 2.33 = 0.24 / 2.33 = \mathbf{10.30\%}$.

CPI 2.33; 5 825 000 cycles; new CPI 2.09; improvement 10.30%. Note the improvement is measured against the **old CPI.**

(e) 24-bit address, 32-byte block, 512-line direct-mapped cache (6 marks) [M3 U3]

GIVEN address = 24 bits · block = 32 bytes · 512 lines · direct mapped

FIND the address format and the line each address maps to

FORMULA $\text{offset} = \log_2(\text{block}) \cdot \text{index} = \log_2(\text{lines}) \cdot \text{tag} = \text{address} - \text{index} - \text{offset} \cdot \text{the index field is the line number}$

1. Offset = $\log_2 32 = \mathbf{5}$ bits. Index = $\log_2 512 = \mathbf{9}$ bits. Tag = 24 – 9 – 5 = **10** bits.
2. Line = (address ÷ 32) mod 512, that is shift right 5 and keep the low 9 bits.
3. A5C3E7 = 10 863 591. Shift right 5: 339 487. mod 512 = **31**.
4. 7F0044 = 8 323 140. Shift right 5: 260 098. mod 512 = **2**.
5. C0FFEE = 12 648 430. Shift right 5: 395 263. mod 512 = **511**.

Format: tag 10 / line 9 / offset 5. Lines: A5C3E7 → 31, 7F0044 → 2, C0FFEE → 511. Here the fields do not fall on hex-digit boundaries (9 index bits is not a whole number of hex digits), so convert to decimal and divide rather than reading digits off, which is the trap this part sets.

(f) Effective addresses (5 marks) [M1 U4]

GIVEN PC = 4000 · R2 = 2600 · address field = 900 · M[900] = 3100, M[2600] = 4900, M[3100] = 750, M[4900] = 88

FIND the effective address and operand in each mode

FORMULA immediate: operand = the field · direct: EA = field · indirect: EA = M[field] · register indirect: EA = register · relative: EA = PC + field

Mode	Effective address	Operand
Immediate	none	900, the field itself
Direct	900	M[900] = 3100
Indirect	M[900] = 3100	M[3100] = 750

Register indirect (R2)	$R2 = 2600$	$M[2600] = 4900$
Relative	$PC + 900 = 4900$	$M[4900] = 88$

Question Two

MODEL ANSWER

Question Two

Write your full answer to **Question Two** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION TWO

(a) Pipeline timing (10 marks) [M2 U3]

GIVEN $k = 5$ stages · stage times 40, 55, 45, 60, 50 ns · latch = 5 ns · $n = 800$ tasks

FIND cycle time, one-task non-pipelined time, ideal speedup, 800-task pipelined and sequential times, throughput

FORMULA cycle = slowest stage + latch · pipelined = $(k + n - 1) \times \text{cycle}$ · sequential = $n \times \Sigma(\text{stage times})$
 · speedup = sequential / pipelined · throughput = $1 / \text{cycle}$ in the steady state

1. Cycle time = slowest stage + latch = $60 + 5 = \mathbf{65 \text{ ns}}$.
2. Non-pipelined, one task = $40 + 55 + 45 + 60 + 50 = \mathbf{250 \text{ ns}}$.
3. Ideal speedup = $250 / 65 = \mathbf{3.85}$ (its ceiling is $k = 5$; the latch and the uneven stages cost the rest).
4. 800 tasks pipelined = $(5 + 800 - 1) \times 65 = 804 \times 65 = \mathbf{52\,260 \text{ ns}}$.
5. 800 tasks sequentially = $800 \times 250 = \mathbf{200\,000 \text{ ns}}$, so the realised speedup is $200\,000 / 52\,260 = \mathbf{3.83}$.
6. Throughput = $800 / 52\,260 \text{ ns} = \mathbf{15.31 \text{ million tasks per second}}$; in the steady state this is one task per cycle, $1 / 65 \text{ ns} = 15.38 \text{ million per second}$.

Cycle 65 ns; one task 250 ns; ideal speedup 3.85; 800 tasks 52 260 ns pipelined against 200 000 ns sequential, a speedup of 3.83; throughput about 15.31 million tasks per second.

(b) The three hazards (4 marks) [M2 U3]

Structural: two instructions need the same hardware in the same cycle (one memory serving a fetch and a data access); cured by duplicating the resource, as with separate instruction and data memories. **Data:** an instruction needs a result the one ahead has not produced yet (read after write); mostly cured by **forwarding**. **Control:** a branch changes what runs next after the pipeline has already fetched the following instructions; cured by resolving the branch early, predicting it, or flushing. Forwarding **cannot** remove the **load-use** data hazard: the loaded value is still being fetched from memory, so exactly one stall remains.

(c) Amdahl's law (6 marks) [M2 U2]

GIVEN $f = 0.75 \cdot s = 8$

FIND the overall speedup, and why it is not 8

FORMULA $\text{speedup} = 1 / ((1 - f) + f/s)$

1. Untouched part: $1 - f = 0.25$, still at full cost.
2. Sped-up part: $f / s = 0.75 / 8 = 0.09375$.
3. Speedup = $1 / (0.25 + 0.09375) = 1 / 0.34375 = \mathbf{2.91}$.

Overall speedup **2.91**, not 8. The 25% that was not improved still takes its original time, and it alone caps the total gain at $1/0.25 = 4$ even if the other 75% were made infinitely fast. That ceiling is the whole content of Amdahl's law.

Question Three

MODEL ANSWER

Question Three

Write your full answer to **Question Three** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION THREE

(a) 8-way set-associative cache design (10 marks) [M3 U3]

GIVEN cache = 32 KB · block = 64 bytes · main memory = 16 MB · $k = 8$ ways

FIND lines, sets, address format, tag directory size

FORMULA address bits = $\log_2(\text{memory})$ · lines = cache / block · sets = lines / k · offset = $\log_2(\text{block})$ · tag = address – set – offset · tag directory = lines × tag bits

1. Address: 16 MB = 2^{24} , so **24-bit** addresses.
2. Lines = 32 KB / 64 B = 32768 / 64 = **512**.
3. Sets = 512 / 8 = **64**.
4. Offset = $\log_2 64 = \mathbf{6}$. Set field = $\log_2 64 = \mathbf{6}$. Tag = 24 – 6 – 6 = **12**.
5. Tag directory = 512 lines × 12 bits = 6144 bits = **768 bytes**.

512 lines in 64 sets of 8. Format **tag 12 / set 6 / offset 6**. Tag directory **768 bytes**, which is the storage overhead the cache pays on top of its 32 KB of data.

(b) AMAT both ways (6 marks) [M3 U3]

GIVEN $h = 0.92 \cdot T_c = 3 \text{ ns} \cdot T_m = 70 \text{ ns}$

FIND AMAT under hierarchical and simultaneous organisation

FORMULA hierarchical = $T_c + (1-h) \times T_m$ · simultaneous = $h \times T_c + (1-h) \times T_m$

1. Miss ratio = $1 - 0.92 = 0.08$.
2. Hierarchical = $3 + 0.08(70) = 3 + 5.6 = \mathbf{8.6 \text{ ns}}$.
3. Simultaneous = $0.92(3) + 0.08(70) = 2.76 + 5.6 = \mathbf{8.36 \text{ ns}}$.

Hierarchical **8.6 ns**, simultaneous **8.36 ns**. The difference is only whether the cache time is paid on every access or only on hits; say which organisation you assumed and you cannot lose the mark.

(c) Locality of reference (4 marks) [M3 U1]

Locality of reference is the tendency of a program to concentrate its memory accesses in a small, predictable set of locations during any short period. **Temporal locality**: a location used now is likely to be used again soon, for example a loop counter read on every iteration, which is why cache keeps recently used data. **Spatial locality**: locations near one just used are likely to be used soon, for example successive elements of an array, which is why cache fetches a whole block of neighbouring words rather than a single one.

Question Four

MODEL ANSWER

Question Four

Write your full answer to **Question Four** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION FOUR

(a) $245 - 178$ by end-around carry (6 marks) [M1 U2]

GIVEN $245 - 178 \cdot n = 10 \text{ bits} \cdot \text{one's complement}$

FIND the difference as a pattern and in decimal

FORMULA $a - b = a + (-b)$ with $-b$ the bitwise inversion; a carry out of the top is **added back** into the lowest bit

1. 245 in 10 bits: $245 = 128 + 64 + 32 + 16 + 4 + 1 = 00 \ 1111 \ 0101$.
2. 178 in 10 bits: $178 = 128 + 32 + 16 + 2 = 00 \ 1011 \ 0010$.
3. -178 in one's complement: invert every bit: $11 \ 0100 \ 1101$.
4. Add: $00 \ 1111 \ 0101 + 11 \ 0100 \ 1101 = 1 \ 00 \ 0100 \ 0010$. A 1 has carried out of the 10th bit.
5. End-around carry: drop it and add it into the lowest bit: $00 \ 0100 \ 0010 + 1 = 00 \ 0100 \ 0011$.
6. Read back: $00 \ 0100 \ 0011 = 64 + 2 + 1 = \mathbf{67}$, and $245 - 178 = 67$. Correct.

$245 - 178 = \mathbf{00 \ 0100 \ 0011} = \mathbf{67}$. The signature of one's complement is that the carry never vanishes, it comes back in at the bottom.

(b) Excess-3 (4 marks) [M1 U2]

GIVEN (i) 507 to encode, (ii) 0111 1001 0110 to decode · code = Excess-3, 4 bits per digit
FIND the code, the decimal, and the self-complementing property
FORMULA encode: digit + 3 in 4 bits · decode: nibble value – 3

- 507: $5+3 = 8 = 1000$; $0+3 = 3 = 0011$; $7+3 = 10 = 1010$. So 507 = **1000 0011 1010**.
- Decode 0111 1001 0110: 0111 = 7, minus 3 = 4; 1001 = 9, minus 3 = 6; 0110 = 6, minus 3 = 3. So it reads **463**.

507 → **1000 0011 1010**; 0111 1001 0110 → **463**. (iii) Excess-3 is **self-complementing**: the code of $(9 - d)$ is the bit-by-bit complement of the code of d . For example 4 is 0111 and 5 is 1000, which are exact complements, so taking the nine's complement of a digit is just inverting its bits. That is what made the code attractive for decimal subtraction hardware.

(c) $Y = (P + Q) \times (R - S) / T$ in four formats (10 marks) [M1 U4]

GIVEN $Y = (P + Q) \times (R - S) / T$ · opcode = 1 byte · each address = 2 bytes
FIND the four programs and their byte totals
FORMULA bytes per instruction = opcode bytes + (address fields × bytes per address); total = instruction count × that

3-address op d,s1,s2	2-address op d,s	1-address accumulator	0-address stack
ADD T1,P,Q SUB T2,R,S MUL T1,T1,T2 DIV Y,T1,T	MOV T1,P ADD T1,Q MOV T2,R SUB T2,S MUL T1,T2 DIV T1,T MOV Y,T1	LOAD P ADD Q STORE T1 LOAD R SUB S MUL T1 DIV T STORE Y	PUSH P PUSH Q ADD PUSH R PUSH S SUB MUL PUSH T DIV POP Y
4 instructions	7 instructions	8 instructions	10 instructions

Format	Bytes per instruction	Total
3-address	$1 + 3 \times 2 = 7$	$4 \times 7 = \mathbf{28}$
2-address	$1 + 2 \times 2 = 5$	$7 \times 5 = \mathbf{35}$
1-address	$1 + 1 \times 2 = 3$	$8 \times 3 = \mathbf{24}$
0-address	3 for PUSH/POP, 1 for an operator	$5 \times 3 + 1 \times 3 + 4 \times 1 = \mathbf{22}$

Totals: **28, 35, 24 and 22 bytes** for 3, 2, 1 and 0 address. The postfix form for the stack machine is **P Q + R S – × T /**. Note the 0-address machine wins here on bytes: its five PUSHes and one POP carry addresses, but its four operators carry none.

Question Five

MODEL ANSWER

Question Five

Write your full answer to **Question Five** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

ANSWER · QUESTION FIVE

(a) C2C60000H in IEEE-754 single precision (6 marks) [M1 U3]

GIVEN pattern = C2C60000H · IEEE-754 single precision · bias 127

FIND the decimal value

FORMULA value = $\pm 1.M \times 2^{(E - 127)}$

1. Expand: C2C60000 = 1100 0010 1100 0110 0000 0000 0000 0000.
2. Split 1 / 8 / 23: sign = 1; E = 1000 0101; M = 100 0110 0000... (rest zeros).
3. Sign 1 means negative.
4. E = 1000 0101 = 133, so e = 133 – 127 = 6.
5. Restore the hidden 1: 1.1000110×2^6 . Move the point 6 places right: 1100011.0.
6. Evaluate $1100011 = 64 + 32 + 2 + 1 = 99$.

C2C60000H = –99. The two marks most often dropped here are forgetting the hidden 1 and forgetting to subtract the bias. Do both, every time.

(b) 21 address lines, 4-byte words (5 marks) [M2 U1]

GIVEN k = 21 address lines · word = 4 bytes · word-addressable

FIND address space, total memory in MB, PC width

FORMULA address space = 2^k · capacity = $2^k \times$ bytes per location · PC width = k

1. Address space = $2^{21} = \mathbf{2\,097\,152}$ locations (words).
2. Each word is 4 bytes, so capacity = $2\,097\,152 \times 4 = 8\,388\,608$ bytes.
3. $8\,388\,608 \div 1024 \div 1024 = \mathbf{8\,MB}$.
4. The PC must hold any address, so it is **21 bits** wide.

2 097 152 words, 8 MB in total, PC 21 bits. Because the machine is word-addressable, each address names a 4-byte word, so the byte capacity is four times the number of locations.

(c) Image size (4 marks) [M1 U2]

GIVEN 8.5 in × 11 in · 200 dpi · black and white, 1 bit per pixel

FIND the memory required, in KB

FORMULA bits = $(w \times dpi) \times (h \times dpi) \times bpp$, then $\div 8$ for bytes and $\div 1024$ for KB

1. Pixels across: $8.5 \times 200 = 1700$. Pixels down: $11 \times 200 = 2200$.
2. Total pixels = $1700 \times 2200 = \mathbf{3\,740\,000}$.
3. At 1 bit per pixel: $3\,740\,000 \text{ bits} \div 8 = \mathbf{467\,500}$ bytes.
4. $467\,500 \div 1024 = \mathbf{456.54 \text{ KB}}$.

$3\,740\,000$ pixels = $467\,500$ bytes $\approx \mathbf{456.54 \text{ KB}}$. Had a colour depth been given, say 24-bit, you would multiply the pixel count by 24 before dividing by 8.

(d) RISC versus CISC (5 marks) [M4 U1]

RISC	CISC
Few, simple, fixed-length instructions	Many, complex, variable-length instructions
Mostly one cycle per instruction	An instruction may take many cycles
Load/store: only those instructions touch memory	Arithmetic may act directly on memory
Many registers, hardwired control, easy to pipeline	Fewer registers, microcoded control, harder to pipeline

Examples: RISC, **ARM** and **MIPS** (also SPARC, RISC-V). CISC, **Intel x86** and **Motorola 68000** (also VAX). The compiler does more work on a RISC machine; the hardware does more on a CISC one.

After you have marked it

YOUR TURN

In Question One (e) you could not read the cache line off the hex digits, though you could in the past papers. Why, and what is the general rule?

ANSWER

Reading fields off hex digits only works when each field is a whole number of hex digits, that is a multiple of **4 bits**. There the offset was 5 bits and the index 9, neither a multiple of 4, so the fields straddle hex-digit boundaries. The general rule: convert to decimal (or to full binary) and use $\text{line} = (\text{address} \div \text{block size}) \bmod \text{number of lines}$. Use the hex shortcut only when the offset and index are multiples of 4 bits, as they were with a 16-word block and 256 lines.